Safe Adaptive Ensemble Reinforcement Learning (SAERL) for Physics-Consistent Fast Charging of Lithium-Ion Battery Packs

Robin K. E. Tau[1,*], Abid Yahya[1], Mmoloki Mangwala[1] ,Nonofo M. J. Ditshego[1],

[1]Department of Electrical and Communications Systems, Botswana International University of Science and Technology, Palapye, Botswana

*Correspondence: tr1800236@studentmail.biust.ac.bw

March 14, 2026

Abstract

Fast charging of lithium-ion battery packs requires simultaneous control of charging speed, safety, and degradation under model uncertainty, sensor noise, and cell-to-cell heterogeneity. This paper presents Safe Adaptive Ensemble Reinforcement Learning (SAERL), a hybrid charging framework that combines three complementary one-step predictors (GRU, MLP, and random forest) through adaptive uncertainty-aware gating, learns a residual policy over a model predictive control (MPC) anchor, and enforces hard voltage, temperature, current, and state-of-charge limits through a safety shield with explicit fallback actions. SAERL is evaluated on 27 realistic charging scenarios constructed from NASA, CALCE, and MATR data across fastest, safe, and long-life objectives, using a pack-level electro-thermal-aging model with explicit heterogeneity, state-estimation error, and sensor noise. Among all compared controllers, including CCCV, MPC, PPO-only, and static-ensemble baselines, SAERL is the only method that achieves non-zero benchmark acceptance, passing 7/27 scenarios (25.9%). It records zero safety failures and zero temperature failures, passes the q-loss criterion in 24/27 cases (88.9%), and reaches a mean time to 80% SoC of 73.3 min with a mean inference latency of 28.7 ms, which is compatible with real-time battery-management operation. Relative to CCCV, SAERL reaches 80% SoC earlier while eliminating hundreds of safety events on average; relative to MPC, it removes all safety events and slightly improves final SoC and q-loss, but remains slower on aggregate. The results show that SAERL is already a safety-dominant and practically deployable hybrid charging architecture, while also making clear that its main remaining limitation is target-reaching performance under domain shift, especially in the NASA aging regime where training support and predictive fidelity are weakest.

Keywords: reinforcement learning, fast charging, lithium-ion batteries, uncertainty quantification, ensemble learning, battery management systems

1 Introduction

Rapid and safe recharge is a central requirement for electrified transport and stationary storage. Lithium ion cells offer high energy density, but they operate within narrow voltage, current, and temperature limits. Violating these limits accelerates degradation and can initiate unsafe failure pathways, including thermal runaway [1, 2]. Fast charging therefore naturally forms a constrained control problem that aims to reduce charge time while respecting safety limits and limiting degradation.

Industrial chargers still rely largely on constant current constant voltage (CCCV) because it is simple, robust, and comparatively easy to certify [2]. In CCCV, a fixed current is applied until a voltage threshold is reached, then the charger holds voltage while the current tapers. This open loop structure is typically conservative, and it cannot explicitly adapt to temperature variation, parameter drift with aging, or cell to cell spread within a pack. These effects become dominant at high C rates, where polarization and heat generation rise sharply [3]. In addition, pack level performance can be limited by the most stressed cell, since cell variability in resistance and capacity drives unequal currents, unequal heating, and unequal aging [4].

Model predictive control (MPC) is an attractive alternative because it optimizes a finite horizon under explicit constraints using a physics based model. However, two issues limit pack level deployment. First, pack safety is fundamentally cell constrained. For a series string with N cells, enforcing per cell voltage and temperature bounds yields at least $2N$ hard constraints per time step and $2NH$ constraints over a horizon of length H. If the controller also tracks per cell electrochemical and thermal states, then both the decision dimension and the constraint set scale with N, and the resulting quadratic program grows quickly in size. This makes real time operation harder as pack size grows, unless substantial simplifications or decentralization are introduced. Second, MPC performance and constraint satisfaction depend on model fidelity. Parameter drift due to aging and temperature, together with cell heterogeneity, can create mismatch that erodes safety margin unless the model is updated online [5].

Reinforcement learning (RL) can learn charging policies that exploit nonlinearities and adapt from data, and recent work shows that adding explicit safety mechanisms can reduce violations during learning and deployment [6]. Yet standard model free RL still faces two practical barriers for batteries. First, it typically requires many interactions, which is expensive or infeasible on physical cells. Second, unsafe exploratory actions can violate safety constraints during training, wasting episodes and risking damage. This motivates hybrid designs that keep learning within safe envelopes while still enabling performance gains beyond classical baselines.

We formulate pack fast charging as a safety critical control problem with hard constraints enforced at the cell level. For all times t and each cell $i \in \{1, \ldots, N\}$, we require

$$V_i(t) \in [V_{\min}, V_{\max}], \qquad T_i(t) \in [T_{\min}, T_{\max}], \tag{1}$$

together with pack level limits on current and total voltage. Safety monitoring and termination

use worst case quantities, including $\max_i V_i(t)$ and $\max_i T_i(t)$, rather than pack averages, so that safety cannot be masked by averaging. This per cell formulation is consistent with the practical reality that the first cell to hit a voltage or temperature limit determines the allowable pack current, especially in the presence of heterogeneity [4].

We propose SAERL, Safe Adaptive Ensemble Reinforcement Learning, a hybrid framework for pack level fast charging under strict per cell safety limits. The main contributions are

1. **Uncertainty aware ensemble adaptation for charging prediction** SAERL integrates complementary predictors, including quantile based neural models and a variance aware tree model, and reweights them online using rolling prediction error and predictive uncertainty. This improves robustness under sensor noise, parameter drift, and cell to cell variability.

2. **Residual learning over a constrained baseline with a differentiable safety shield** SAERL learns a residual correction on top of a baseline constrained controller to accelerate learning while keeping exploration near a safe operating region. Candidate actions pass through a differentiable safety shield that checks per cell hard constraints on voltage and temperature, plus operational bounds on current and state of charge, before application. Unsafe actions are rejected before the environment is stepped.

3. **Pack level evaluation under heterogeneity and worst cell metrics** We evaluate SAERL on a pack model with explicit cell to cell variation in capacity and resistance. Performance is reported using worst cell voltage and temperature trajectories, enabling direct analysis of imbalance and safety margins that single cell studies cannot capture [4].

4. **Cross regime scenario evaluation with public cycling data** We construct fast charging scenarios from public sources that span different ambient conditions and cycling regimes, including the NASA prognostics repository data [7] and the MATR related cycle life study [8]. To avoid ambiguous mixing, all scenarios are mapped to a common cell model through parameter identification, ensuring consistent state definitions and constraint thresholds across domains.

2 Background and Related Work

2.1 Charging Strategies

2.1.1 Constant Current Constant Voltage (CCCV)

Constant current constant voltage (CCCV) remains the industrial benchmark for lithium ion battery charging because it is simple, reliable, and straightforward to implement and certify [9, 10]. CCCV consists of two phases. In the constant current (CC) phase, the charger applies a fixed current I_{cc} until the terminal voltage reaches the upper cutoff $V_{\max}$. In the constant voltage (CV) phase, the charger holds $V(t) = V_{\max}$ while the current naturally tapers, and charging terminates when $I(t)$ falls below a threshold I_{term} such as C/20 [11].

A compact description is

$$I(t) = I_{\text{cc}}, \quad \text{while } V(t) < V_{\max}. \tag{2}$$

$$V(t) = V_{\max}, \quad \text{while } I(t) > I_{\text{term}}. \tag{3}$$

During the CV phase, the current taper can be related to the difference between the imposed terminal voltage and the evolving open circuit voltage. Under a simplified resistance dominated approximation,

$$I(t) \approx \frac{V_{\max} - V_{\text{oc}}(t)}{R_0(t)}. \tag{4}$$

This explains the well known long tail near end of charge as $V_{\text{oc}}(t)$ approaches $V_{\max}$ and as $R_0(t)$ increases with aging and low temperature [9, 10].

CCCV has clear practical advantages. It is open loop and does not require explicit estimation of internal states such as state of charge (SoC) or state of health (SoH), and it does not depend on identifying or updating model parameters online. This makes it robust across cell formats and operating conditions, especially in low cost charger implementations.

However, CCCV is not time optimal for fast charging. High I_{cc} often drives the terminal voltage to $V_{\max}$ early due to polarization, forcing an extended CV phase where the delivered charge per unit time steadily decreases. As a result, increasing I_{cc} yields diminishing returns in total charge time [9]. More importantly, the same fixed $(I_{\text{cc}}, V_{\max})$ pair can be benign for a fresh warm cell yet risky for a cold or aged cell, since higher resistance and slower diffusion amplify overpotential and local lithium plating propensity at high rates [9, 10]. These limitations motivate baseline alternatives such as multistage and pulse charging, which seek to reduce the CV tail and mitigate stress, though they still rely on rule design and may remain chemistry and condition dependent [9, 12].

2.1.2 Model Predictive Control (MPC)

Model predictive control (MPC) replaces fixed rules with repeated finite horizon optimization under an explicit battery model. At each sampling instant k, MPC computes a current sequence that minimizes a cost while enforcing safety limits over a horizon of length H

$$\min_{\{I_{k+j}\}_{j=0}^{H-1}} \sum_{j=0}^{H-1} \ell(x_{k+j}, I_{k+j}) + \Phi(x_{k+H}). \tag{5}$$

subject to

$$x_{k+j+1} = f(x_{k+j}, I_{k+j}), \quad j = 0, \ldots, H-1, \tag{6}$$

$$V_{\min} \le V_{k+j} \le V_{\max}, \quad j = 0, \ldots, H-1, \tag{7}$$

$$T_{\min} \le T_{k+j} \le T_{\max}, \quad j = 0, \ldots, H-1, \tag{8}$$

$$0 \le I_{k+j} \le I_{\max}, \quad j = 0, \ldots, H-1. \tag{9}$$

Here x_k contains the states used by the embedded model, for example SoC, polarization states, and thermal states, and $\ell(\cdot)$ can penalize time, current stress, and degradation related surrogates [11, 13].

The main advantage of MPC is systematic constraint handling. Voltage, temperature, and current bounds can be embedded as hard constraints, and the predictive structure allows the controller to preemptively reduce current before violating limits rather than reacting after the fact.

Pack level deployment introduces two recurring issues. First, the optimization size grows with pack granularity. If a series string has N cells and the safety specification is per cell, then enforcing bounds on each V_i and T_i yields at least $2N$ inequality constraints per step and $2NH$ constraints over the horizon. If the model tracks per cell polarization and thermal dynamics, the state dimension also increases with N, increasing solve times and making real time implementation harder as pack size grows [13, 14]. This scaling argument can be summarized as

$$m_{\text{ineq}}(N, H) \geq 2NH, \tag{10}$$

where m_{ineq} is the number of per step inequality constraints aggregated over the horizon.

Second, MPC performance depends on model fidelity. Equivalent circuit and reduced order electrochemical models are approximations and their parameters drift with temperature and aging. Model mismatch can be written as

$$x_{k+1} = f_{\text{true}}(x_k, I_k) = f_{\text{model}}(x_k, I_k) + w_k, \tag{11}$$

where w_k captures unmodeled dynamics and parameter errors. In charging, even modest errors in internal resistance can shift predicted terminal voltage and shrink the effective safety margin. For example, if R_0 is overestimated, then the predicted terminal voltage

$$\widehat{V}_t = V_{\text{oc}} - \widehat{R}_0 I \tag{12}$$

suggests earlier saturation and drives unnecessary current derating, increasing charge time [11, 13]. These limitations motivate hybrid approaches that retain MPC as a safe baseline while learning residual corrections and uncertainty handling mechanisms, aligned with the proposed method in this paper.

2.2 Reinforcement Learning for Battery Control

Reinforcement learning (RL) offers a data driven route to learn charging policies directly from interaction, without requiring an explicit closed form electrochemical model. Fast charging can be posed as a Markov decision process (MDP) with state $s_t \in \mathcal{S}$, action $a_t \in \mathcal{A}$, transition kernel $P(s_{t+1} \mid s_t, a_t)$, reward $r(s_t, a_t)$, and discount factor $\gamma \in (0, 1]$. In battery charging, the state commonly aggregates measurable or estimated quantities such as SoC, terminal voltage, temperature, and short histories of these signals, while the action is typically the commanded charge current or

an incremental adjustment to a baseline current.

2.2.1 Model free reinforcement learning

Model free RL methods learn a policy $\pi_\theta(a \mid s)$ directly from reward signals. The standard objective maximizes the expected discounted return

$$J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \gamma^t \, r(s_t, a_t) \right], \tag{13}$$

where $\tau = (s_0, a_0, s_1, a_1, \dots)$ is a trajectory induced by the policy. Value based approaches such as deep Q networks and policy gradient methods such as PPO have been used in battery energy management and charging control settings [15–17]. For PPO, policy updates are commonly computed by maximizing the clipped surrogate objective

$$L_{\mathrm{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \, \hat{A}_t, \mathrm{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon) \, \hat{A}_t \right) \right], \tag{14}$$

with probability ratio

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t \mid s_t)}, \tag{15}$$

advantage estimate $\hat{A}_t$, and clipping hyperparameter ϵ [17]. Despite flexibility, model free RL remains difficult to apply on physical cells because learning often requires very large numbers of episodes, while each charging episode is time consuming and contributes measurable aging. In addition, unconstrained exploration can violate voltage and temperature limits, which is unacceptable for safety critical battery operation.

2.2.2 Model based reinforcement learning

Model based RL improves sample efficiency by learning a predictive dynamics model and using it for planning or policy optimization. A common one step stochastic predictor is

$$\hat{s}_{t+1} = f_\phi(s_t, a_t) + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, \Sigma_\phi(s_t, a_t)), \tag{16}$$

where f_ϕ is a learned function approximator and Σ_ϕ captures aleatoric uncertainty. Planning with learned probabilistic dynamics and short horizon rollouts has been shown to reduce real interaction requirements in general control tasks [18]. For batteries, recent work also explores deep RL charging strategies that incorporate aging models directly in the learning loop, highlighting both the promise and the difficulty of achieving robust, transferable policies across conditions [19]. The key risk is model bias, where prediction errors induce unsafe actions or overly aggressive currents when deployed outside the training distribution.

2.2.3 Safe reinforcement learning with safety layers and shielding

Safety constrained charging can be written using inequality constraints $g(s_t, a_t) \leq 0$ that encode voltage, temperature, current, and SoC limits. A widely used safety layer is action projection, which modifies the RL proposed action a_t^{RL} by solving

$$a_t^{\mathrm{safe}} = \arg\min_{a \in \mathcal{A}} \left\| a - a_t^{\mathrm{RL}} \right\|_2^2 \quad \text{subject to} \quad g(s_t, a) \leq 0. \tag{17}$$

For battery fast charging, Wei et al. applied this idea with linearized constraints and quadratic programming, improving safety while adding per step optimization overhead [20]. Pack level deployment can further increase cost since per cell voltage and temperature limits yield constraint sets that grow with the number of series cells.

An alternative is shielding, where a separate safety mechanism evaluates candidate actions using simplified predictive checks and overrides unsafe actions with a known safe fallback. Shielding can provide strong safety guarantees during both training and execution when the shield is constructed to conservatively enforce constraints [21]. This perspective motivates lightweight shields that avoid solving a full quadratic program online while still preventing unsafe transitions.

2.2.4 Residual policy learning over a baseline controller

Residual RL reduces exploration burden by learning corrections around a baseline controller $\pi_{\mathrm{base}}(s_t)$, such as CCCV or MPC. The applied action is

$$a_t = \pi_{\mathrm{base}}(s_t) + \delta_t, \qquad \delta_t \sim \pi_\theta^{\mathrm{res}}(\cdot \mid s_t), \tag{18}$$

so the agent searches in a smaller neighborhood around a reasonable default, which can accelerate learning and improve stability [22]. In safety critical charging, residual learning pairs naturally with shielding, since the baseline provides a safe backbone and the shield rejects residuals that would violate per cell limits.

The above limitations motivate hybrid designs that combine a strong baseline, safe exploration, and robustness to sensing errors and cell variability.

2.3 Uncertainty Quantification and Ensemble Methods

Fast charging is inherently safety critical, and the controller must act on predictions that can be unreliable due to sensing noise, cell variability, aging, and operating points not covered by training data. A useful distinction is between aleatoric uncertainty, which reflects irreducible randomness such as sensor noise, and epistemic uncertainty, which reflects lack of knowledge due to limited or shifted data [23]. In safety constrained control, epistemic uncertainty is particularly important because it can indicate out of distribution conditions where aggressive actions are risky.

Deep ensembles A practical and well calibrated approach to epistemic uncertainty uses deep ensembles [24]. Consider an ensemble of M predictors $\{f_m(x)\}_{m=1}^{M}$ trained with different initializations or data shuffles. The ensemble predictive mean is

$$\hat{\mu}(x) = \frac{1}{M} \sum_{m=1}^{M} f_m(x). \tag{19}$$

A simple estimator of epistemic uncertainty is the disagreement variance

$$\hat{\sigma}^2(x) = \frac{1}{M-1} \sum_{m=1}^{M} \big(f_m(x) - \hat{\mu}(x)\big)^2. \tag{20}$$

Large $\hat{\sigma}^2(x)$ indicates low confidence and motivates conservative actions or stronger safety margins.

Probabilistic ensembles in model based RL Ensembles also underpin sample efficient model based RL, where the dynamics are learned and used for planning [18]. A common formulation uses an ensemble of probabilistic models that predict a distribution over the next state

$$\hat{p}_m(s_{t+1} \mid s_t, a_t) = \mathcal{N}\big(\mu_m(s_t, a_t), \Sigma_m(s_t, a_t)\big), \tag{21}$$

where Σ_m captures aleatoric uncertainty and disagreement across μ_m captures epistemic uncertainty. Trajectory sampling across models reduces brittleness to model error, though online planning can still be too costly for embedded BMS constraints.

Gaussian process regression for battery prognostics In battery health and aging prediction, Gaussian process regression is widely used because it provides principled predictive intervals [25]. A Gaussian process prior is

$$f(x) \sim \mathcal{GP}\big(m(x), k(x, x')\big), \tag{22}$$

and with training targets y and kernel matrix K the predictive mean and variance at x_* are

$$\mu(x_*) = k_*^\top \big(K + \sigma_n^2 I\big)^{-1} y, \tag{23}$$

$$\sigma^2(x_*) = k(x_*, x_*) - k_*^\top \big(K + \sigma_n^2 I\big)^{-1} k_*. \tag{24}$$

Despite their attractive uncertainty properties, standard GP methods scale poorly with dataset size, which limits direct online use when transition data becomes large.

Quantile based uncertainty Distribution free predictive intervals can be obtained via quantile regression [26]. For a quantile level $\tau \in (0, 1)$, the pinball loss is

$$\mathcal{L}_\tau(y, \hat{y}) = \begin{cases} \tau\,(y - \hat{y}), & y \geq \hat{y}, \\ (1 - \tau)\,(\hat{y} - y), & y < \hat{y}. \end{cases} \tag{25}$$

Training a neural predictor to output multiple quantiles yields a prediction interval whose width is an uncertainty proxy. Another scalable approximate Bayesian alternative is Monte Carlo dropout, which interprets dropout at inference time as approximate Bayesian inference [27].

Adaptive ensemble weighting Many ensembles use static averaging, which is suboptimal when operating conditions change due to temperature shifts, aging, or distribution drift. A simple adaptive weighting rule based on recent error and uncertainty histories can be written as

$$w_{m,t} = \frac{\exp(-\alpha e_{m,t} - \beta u_{m,t})}{\sum_{j=1}^{M} \exp(-\alpha e_{j,t} - \beta u_{j,t})}, \tag{26}$$

where $e_{m,t}$ is a rolling prediction error metric and $u_{m,t}$ is an uncertainty proxy for model m. More generally, mixture of experts style gating learns a context dependent weighting function, often implemented with a small neural gate [28]. Let z_t collect features such as recent errors, uncertainties, and operating point indicators, then

$$w_t = \mathrm{softmax}(g_\psi(z_t)), \tag{27}$$

which permits rapid shifts in reliance when one expert becomes unreliable.

3 Problem Formulation

3.1 Motivation

Fast charging of a lithium ion battery pack is a safety critical control problem. The controller must deliver as much charge as possible in the shortest time while ensuring that every individual cell remains within strict voltage and temperature limits. Even brief excursions beyond these limits accelerate degradation and can trigger unsafe failure modes. The pack setting makes this harder because cell to cell variations in capacity, resistance, and thermal coupling produce unequal voltage responses and unequal heat generation under the same series current. Pack level averages can therefore look safe while an outlier cell is already near a hard limit.

We formulate the task as a constrained Markov decision process defined by the tuple $(\mathcal{S}, \mathcal{A}, P, r, c, \gamma)$. The environment is a high fidelity simulation of a series connected pack with $N = 10$ cells. The objective is to move the pack from an initial state of charge in the range 10% to 30% to a target of 80% as quickly as possible while satisfying hard safety constraints for every cell at each discrete time step t with sampling interval $\Delta t = 1\,\mathrm{s}$. Safety is monitored using worst cell quantities such as $V_{\max,t} = \max_i V_{i,t}$ and $T_{\max,t} = \max_i T_{i,t}$ so that no single cell can be hidden by pack averaging.

3.2 Symbols and indices

Table 1: Key symbols used in the formulation

Symbol	Meaning
i	cell index, $i \in \{1, \ldots, N\}$
t	discrete time index
Δt	sampling interval
$z_{i,t}$	true state of charge of cell i
$\hat{z}_{i,t}$	estimated state of charge of cell i
$\bar{z}_t$	average estimated state of charge across cells
$V_{i,t}$	terminal voltage of cell i
$T_{i,t}$	temperature of cell i
I_t	pack current, positive for discharge and negative for charge
$I_{\max}$	maximum magnitude of charge current
$V_{\min}, V_{\max}$	voltage safety bounds
$T_{\max}$	temperature safety bound

3.3 Battery and pack model

A controller intended for deployment must be evaluated in an environment that reproduces the dynamics that actually constrain fast charging. These include electrochemical polarization, thermal buildup, aging, and heterogeneity. A model that ignores any of these can overestimate achievable charging speed or underestimate risk. We therefore model each cell with an electro thermal equivalent circuit coupled to a semi empirical aging law, then aggregate cells into a physically consistent pack with parameter variation and thermal coupling.

3.3.1 Single cell electro thermal dynamics

Each cell is represented by a second order Thevenin model, consisting of an ohmic resistance R_0 and two RC polarization pairs (R_1, C_1) and (R_2, C_2). This choice balances fidelity and computational tractability. The ohmic term captures the instantaneous voltage drop that determines proximity to the voltage limit during high current charging. The RC terms capture charge transfer and diffusion polarization which create the delayed voltage response that becomes critical near the end of charge.

State of charge evolves via Coulomb counting

$$\frac{dz_i}{dt} = -\frac{\eta\, I}{Q_{\text{eff},i}\, 3600}, \tag{28}$$

where $\eta = 0.995$ is Coulombic efficiency, I is the series current shared by all cells, and $Q_{\text{eff},i}$ is the effective capacity that decreases with aging. This equation is the standard BMS backbone, so learning on top of it encourages deployment realism.

The polarization voltages $v_{1,i}$ and $v_{2,i}$ follow

$$\frac{dv_{k,i}}{dt} = -\frac{v_{k,i}}{R_{k,i}C_{k,i}} + \frac{I}{C_{k,i}}, \qquad k \in \{1,2\}, \tag{29}$$

which captures the fact that polarization builds under current and relaxes when current is reduced. This is precisely the mechanism that a controller exploits when tapering current near voltage limits.

Terminal voltage is computed as

$$V_i = V_{\mathrm{ocv}}(z_i) - I\,R_{0,i}^{\mathrm{eff}} - v_{1,i} - v_{2,i}, \tag{30}$$

where $V_{\mathrm{ocv}}(z_i)$ is interpolated from a manufacturer lookup table spanning 3.0 V at 0% SoC to 4.2 V at 100% SoC. The effective ohmic resistance

$$R_{0,i}^{\mathrm{eff}} = R_{0,i} + R_{0,i}^{\mathrm{growth}} \tag{31}$$

includes aging induced growth. This coupling is essential because an aged cell experiences a larger voltage rise under the same current, which directly tightens the feasible charging envelope.

Temperature follows a lumped thermal model

$$\frac{dT_i}{dt} = \frac{Q_{\mathrm{gen},i} - Q_{\mathrm{diss},i}}{C_{\mathrm{th}}}, \qquad Q_{\mathrm{gen},i} = I^2 R_{0,i}^{\mathrm{eff}}, \qquad Q_{\mathrm{diss},i} = hA\,(T_i - T_{\mathrm{amb}}), \tag{32}$$

with $C_{\mathrm{th}} = 75\,\mathrm{J/^\circ C}$ and $hA = 0.5\,\mathrm{W/^\circ C}$. The dominant heat source during high rate charging is resistive heating, so this model captures the main thermal driver while remaining fast enough for repeated rollouts during training and shielding.

3.3.2 Aging model and parameter coupling

Fast charging that ignores aging can look optimal in a single episode yet produce unacceptable life loss over repeated cycles. We therefore include an aging model that updates both capacity and resistance to create a realistic feedback loop. Capacity fade reduces usable charge acceptance. Resistance growth increases heating and accelerates future aging.

We model loss of capacity through a stress factor law

$$\frac{dQ_{\mathrm{loss},i}}{dt} = k_1\, S_{I,i}\, S_{T,i}\, S_{z,i}, \tag{33}$$

where the stress factors represent current, temperature, and state of charge effects

$$S_{I,i} = \frac{|I|}{Q_{\mathrm{nom},i}}, \qquad S_{T,i} = \exp\big(k_2(T_i - T_{\mathrm{ref}})\big), \qquad S_{z,i} = \exp\big(k_3 z_i\big), \tag{34}$$

with $k_2 = 0.05$, $T_{\mathrm{ref}} = 25\,^\circ\mathrm{C}$, and $k_3 = 0.1$. These forms reflect well known qualitative trends. Higher C rate increases plating and mechanical stress. Higher temperature accelerates side reactions.

High SoC increases parasitic oxidation and electrolyte decomposition.

We map accumulated loss to a normalized fade variable

$$\mathrm{fade}_i = \min\left(\frac{Q_{\mathrm{loss},i}}{0.2Q_{\mathrm{nom},i}},\, 0.8\right), \tag{35}$$

then update effective parameters

$$Q_{\mathrm{eff},i} = Q_{\mathrm{nom},i}(1-\mathrm{fade}_i), \qquad R_{0,i}^{\mathrm{growth}} = 2R_{0,i}\,\mathrm{fade}_i. \tag{36}$$

This ensures that aging changes both SoC dynamics and thermal dynamics, which forces the controller to consider long term stress rather than only short term progress.

3.3.3 Pack heterogeneity and thermal coupling

Real packs are heterogeneous. Cells with slightly lower capacity or higher resistance reach voltage limits earlier and heat faster, which often dictates the pack level limit. If training assumes identical cells, the learned policy can be brittle when deployed. We therefore inject realistic variation through correlated sampling

$$Q_{\mathrm{nom},i} \sim \mathcal{N}(2.5\,\mathrm{Ah},\, 5\%)\,, \qquad R_{0,i} \sim \mathcal{N}(20\,\mathrm{m}\Omega,\, 20\%)\,, \tag{37}$$

with correlation $\rho_{QR} = -0.3$ so that lower capacity tends to co occur with higher resistance. Initial SoC is perturbed as

$$z_{i,0} = z_0 + \xi_i, \qquad \xi_i \sim \mathcal{U}(-0.05,\, 0.05), \tag{38}$$

which creates imbalance from the start, as is typical after storage or prior usage. Thermal coupling between adjacent cells is modeled through nearest neighbor conduction, producing spatial temperature gradients that affect both safety and aging.

3.3.4 BMS realism through sensor noise and estimation

Controllers do not have access to perfect states. Voltage and temperature sensors exhibit noise, and SoC is estimated rather than measured. To avoid learning policies that exploit simulator access, we corrupt measurements with Gaussian noise

$$\tilde{V}_{i,t} = V_{i,t} + \nu_{i,t}^V, \qquad \nu_{i,t}^V \sim \mathcal{N}(0, \sigma_V^2), \qquad \sigma_V = 10\,\mathrm{mV}, \tag{39}$$

$$\tilde{T}_{i,t} = T_{i,t} + \nu_{i,t}^T, \qquad \nu_{i,t}^T \sim \mathcal{N}(0, \sigma_T^2), \qquad \sigma_T = 0.5\,^\circ\mathrm{C}. \tag{40}$$

SoC estimation follows noisy Coulomb counting

$$\hat{z}_{i,t+1} = \mathrm{clip}\left(\hat{z}_{i,t} - \eta_i \frac{I_t \Delta t}{C_i\, 3600} + \epsilon_t,\, 0,\, 1\right), \qquad \epsilon_t \sim \mathcal{N}(0, \sigma_{\mathrm{soc}}^2), \qquad \sigma_{\mathrm{soc}} = 0.01. \tag{41}$$

This forces the learned controller to remain robust to estimation error, which is a practical deployment requirement.

3.4 Constrained Markov Decision Process(CMDP) specification

3.4.1 State space

The observed state s_t must be informative enough for safe decisions while remaining measurable in a real BMS. We therefore construct s_t from noisy estimates and worst cell summaries. The state includes progress, safety margins, imbalance, and recent actuation history, plus uncertainty features produced by the predictive ensemble

$$s_t = \Big[\bar{z}_t,\, V_{\min,t},\, V_{\max,t},\, T_t,\, \sigma_{z,t},\, I_{t-1},\, I_{t-1}^2,\, u_t^{\text{gru}},\, u_t^{\text{mlp}},\, u_t^{\text{rf}}\Big]. \tag{42}$$

Here $V_{\min,t} = \min_i \tilde{V}_{i,t}$ and $V_{\max,t} = \max_i \tilde{V}_{i,t}$ provide early warning of voltage saturation. The term I_{t-1}^2 is included because resistive heating scales with current squared, so it provides a direct proxy for imminent thermal rise. All continuous features are normalized to $[0, 1]$ using running statistics collected during training. A sliding window of the last $L = 20$ steps is used as policy input to capture thermal inertia and polarization memory.

3.4.2 Action space

The action is a normalized current command $a_t \in [-1, 1]$ mapped to physical current as

$$I_t = -\frac{1}{2} I_{\max}\,(a_t + 1). \tag{43}$$

With this convention, $a_t = -1$ yields $I_t = 0$ as a safe fallback, while $a_t = 1$ yields $I_t = -I_{\max}$ as maximum charging current. We set $I_{\max}$ to correspond to a $5C$ limit, consistent with aggressive fast charging scenarios. Actions are held constant over each 1 s step, consistent with typical BMS control update rates.

3.4.3 Reward design

Fast charging is inherently multi objective. Higher current reduces time but increases heating, imbalance growth, and aging. The reward must therefore encode a practical trade off while remaining dense enough for stable learning. We use

$$r_t = 10\,\bar{z}_t - 0.01 - 0.1\max(0, T_t - 40)^2 - 0.05\,(1000\,\Delta V_t) - 1.0\,n_{\text{safety},t} + 50\,\mathbf{1}\big[\bar{z}_t \geq 0.8\big], \tag{44}$$

where $\Delta V_t = V_{\max,t} - V_{\min,t}$ is voltage spread and $n_{\text{safety},t}$ counts the number of per cell violations at time t. The time penalty assigns a cost to each step so shorter episodes are preferred. The quadratic temperature penalty reflects the nonlinear increase of degradation risk with temperature, so the controller is discouraged from hovering near thermal limits. The imbalance term encourages

keeping cells aligned, which improves both safety margin and charge acceptance for the worst cell. The completion bonus stabilizes the objective by explicitly rewarding reaching the target SoC.

3.4.4 Hard safety constraints, cost, and termination

Hard limits must be satisfied for every cell at every time step

$$V_{\min} \leq V_{i,t} \leq V_{\max}, \qquad T_{i,t} \leq T_{\max}, \qquad \forall i, \forall t. \tag{45}$$

These constraints represent non negotiable safety requirements rather than soft preferences. We therefore introduce a safety cost

$$c_t = \mathbf{1}\Big[V_{\max,t} > V_{\max} \text{ or } T_{\max,t} > T_{\max}\Big], \tag{46}$$

and require the expected cumulative cost to be zero under the deployed policy. Episodes terminate when $\bar{z}_t \geq 0.8$. Episodes also terminate under critical violations, such as pack level over voltage or under voltage, or sustained thermal exceedance. This separation is useful in practice because it allows transient penalties to shape learning while still enforcing strict termination for conditions that would trigger a real system shutdown.

4 Methodology: Safe Adaptive Ensemble Reinforcement Learning(SAERL)

4.1 Design rationale

A pure reinforcement learning controller trained by trial and error is unsuitable for this problem because unsafe exploration is unacceptable. A pure model based controller such as MPC provides structure and constraint handling, but its performance degrades under model mismatch, aging drift, and unmodeled heterogeneity. It can also become computationally expensive as pack complexity increases. These limitations motivate a hybrid design that uses model based control as a safe starting point while learning targeted improvements and adapting to uncertainty online.

Safe Adaptive Ensemble Reinforcement Learning integrates four modules. A predictive ensemble provides short horizon forecasts with uncertainty. An adaptive gating network fuses experts based on recent accuracy and current uncertainty. A residual policy learns corrections to an MPC baseline action. A safety shield filters candidate actions through a one step physics rollout to guarantee feasibility.

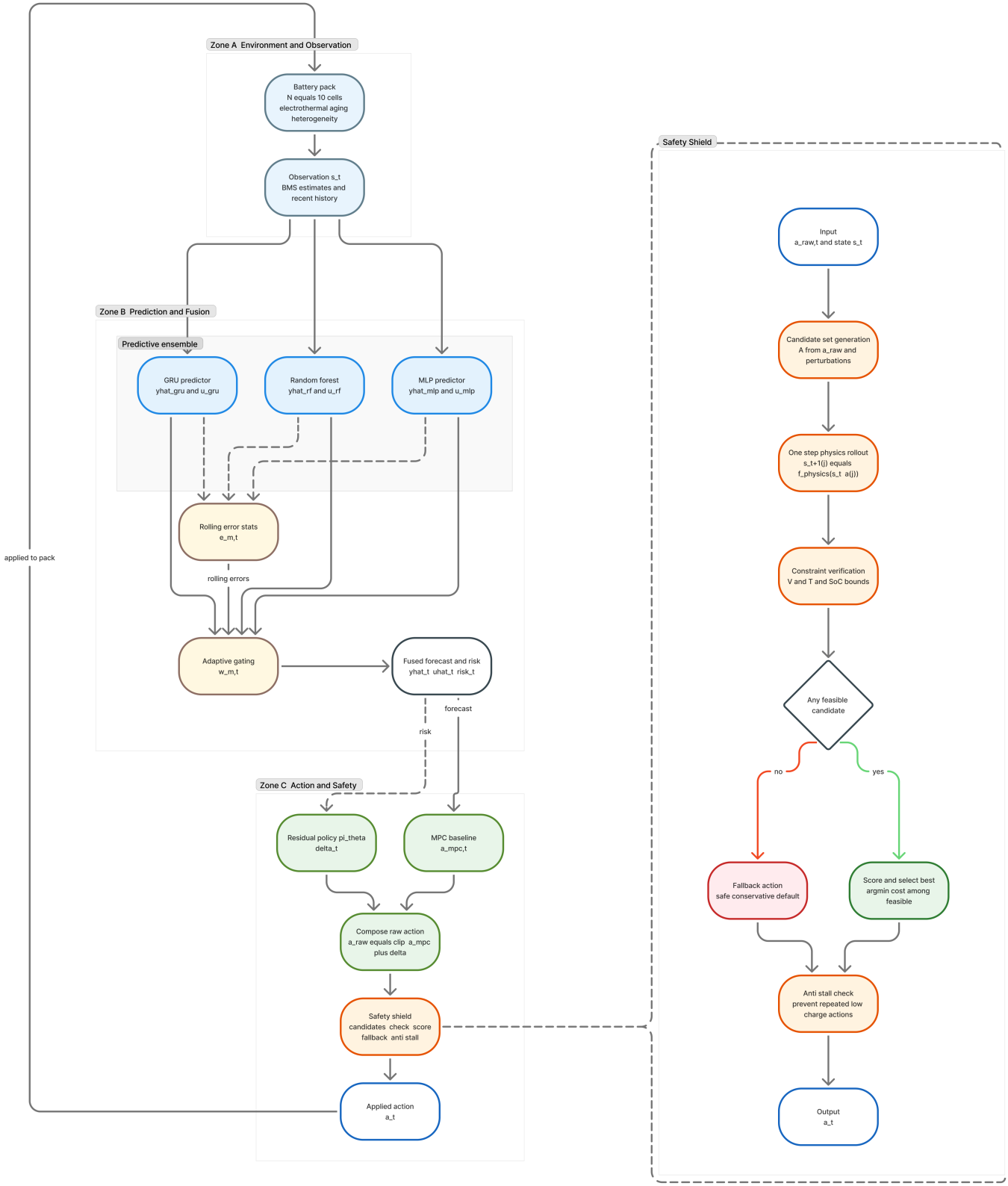


Figure 1: SAERL architecture for online fast-charging control.

4.2 Predictive ensemble and uncertainty

Point predictions alone are risky because they do not tell the controller when it should be cautious. We therefore use a diverse ensemble to capture different inductive biases and to expose epistemic uncertainty through disagreement.

The ensemble contains three predictors. A GRU with quantile outputs captures temporal dependencies in polarization and temperature. An MLP with quantile outputs captures nonlinear feature interactions without relying on recurrent memory. A random forest regressor adds robustness to outliers and provides variance across trees as a nonparametric uncertainty estimate.

Each model predicts next step targets that directly affect safety and control, namely $\bar{z}_{t+1}$, $V_{\min,t+1}$, T_{t+1}, and ΔV_{t+1}. For the GRU and MLP, we train quantiles $\tau \in \{0.1, 0.5, 0.9\}$ using the pinball loss

$$\mathcal{L}_\tau(y, \hat{y}) = \begin{cases} \tau(y - \hat{y}), & y \geq \hat{y}, \\ (1-\tau)(\hat{y} - y), & y < \hat{y}. \end{cases} \tag{47}$$

The prediction interval width

$$u_t^m = q_{0.9}^m(t) - q_{0.1}^m(t) \tag{48}$$

acts as an uncertainty proxy. It grows when dynamics become harder to predict, such as near voltage saturation, under high C rate, or under shifting resistance due to aging. For the random forest, we compute $\hat{\mu}$ and $\hat{\sigma}$ from tree predictions and set

$$q_{0.1}^{\text{rf}} = \hat{\mu} - 1.64\hat{\sigma}, \qquad q_{0.9}^{\text{rf}} = \hat{\mu} + 1.64\hat{\sigma}, \qquad u_t^{\text{rf}} = q_{0.9}^{\text{rf}} - q_{0.1}^{\text{rf}}. \tag{49}$$

This gives all experts a comparable uncertainty representation.

4.3 Adaptive gating network

A static average of expert predictions is suboptimal because each model dominates in different regimes. The GRU may excel during smooth charging trajectories. The random forest may be more reliable under noisy measurements. Aging drift can also change the relative ranking of experts over time. We therefore learn a gate that allocates trust online based on context, recent error, and uncertainty.

Let $\hat{y}_{m,t}$ be expert m prediction and $u_{m,t}$ its uncertainty proxy. We maintain a rolling mean absolute error estimate over the last $K = 10$ steps

$$e_{m,t} = \frac{1}{K} \sum_{k=0}^{K-1} \left| y_{t-k} - \hat{y}_{m,t-k} \right| . \tag{50}$$

The gate takes as input the current state features, the current candidate action, the vector of errors $(e_{m,t})$, and uncertainties $(u_{m,t})$. A two layer MLP outputs logits $\ell_{m,t}$ and weights are obtained by

softmax

$$w_{m,t} = \frac{\exp(\ell_{m,t})}{\sum_j \exp(\ell_{j,t})}, \qquad w_{m,t} \geq 0, \qquad \sum_m w_{m,t} = 1. \tag{51}$$

The fused prediction and fused uncertainty are

$$\hat{y}_t = \sum_m w_{m,t}\, \hat{y}_{m,t}, \qquad \hat{u}_t = \sum_m w_{m,t}\, u_{m,t}. \tag{52}$$

Weights are refreshed every 50 steps to reduce compute overhead, which is important for real time feasibility.

4.4 Residual policy over an MPC baseline

Learning a full charging policy from scratch requires extensive exploration, which is unsafe and sample inefficient. We therefore learn a residual correction to an MPC baseline action. The baseline provides a conservative, model informed default. The residual focuses learning on the gap between baseline behavior and the best achievable charging speed under the true environment.

Let $a_{\mathrm{mpc},t}$ be the normalized MPC action. The policy outputs a residual δ_t and composes

$$a_{\mathrm{raw},t} = \mathrm{clip}(a_{\mathrm{mpc},t} + \delta_t,\, -1,\, 1)\,, \qquad \delta_t \sim \pi_\theta(\cdot \mid s_t). \tag{53}$$

The residual policy π_θ is Gaussian with mean produced by a two layer MLP of width 128 and a learned standard deviation. The input includes the flattened sliding window of the last L feature vectors, plus $a_{\mathrm{mpc},t}$ and the SoC gap to target. This gives the policy both temporal context and the baseline suggestion, which reduces variance and accelerates learning.

4.5 Safety shield using one step physics rollout

Even with residual learning, a stochastic policy can propose unsafe actions, especially under distribution shift or early in training. Penalizing violations in the reward is insufficient because it allows violations to occur. We therefore enforce safety through a shield that filters actions before they are applied.

4.5.1 Candidate generation

Given $a_{\mathrm{raw},t}$, the shield generates a finite candidate set around the proposed residual. We include 9 linearly spaced residual perturbations within ± 0.35 of the sampled residual, the baseline MPC action as an anchor, a CCCV anchor, and the maximum action $a = 1$ so that full rate is always considered when safe. Each candidate a is mapped to physical current using the same affine transform as the environment.

4.5.2 One step feasibility check

For each candidate, the shield performs a one step rollout of the full physics model. The complete pack state is saved, the candidate current is applied for one step, constraints are checked, and the original state is restored. A candidate is admissible only if all per cell constraints are satisfied at $t+1$

$$\hat{V}_{i,t+1} \le V_{\max}, \qquad \hat{T}_{i,t+1} \le T_{\max}, \qquad \hat{z}_{i,t+1} \le 1, \qquad \forall i, \tag{54}$$

together with pack level limits and a voltage spread margin of 95 mV to avoid operating too close to the boundary under sensor noise. This check turns safety into a verified property of the applied action rather than a learned preference.

4.5.3 Cost based ranking of safe candidates

When multiple candidates are safe, the shield selects the one that best balances speed and stress using a cost score that mirrors the intended trade offs. For a candidate action a, define predicted SoC gain $\Delta\widehat{\mathrm{SoC}}(a)$, predicted temperature excess $\Delta\hat{T}^{+}(a) = \max(0, \hat{T}(a) - T_{\mathrm{soft}})$, and predicted imbalance change $\Delta\hat{V}_{\mathrm{imb}}(a)$. We also define a degradation proxy consistent with the stress factor structure

$$D_{\mathrm{proxy}}(a) = \mathrm{Cr}(a)^2 \exp\left(\frac{\max(0, \hat{T}(a) - 25)}{18}\right) \exp(0.8\,\hat{z}(a))\,, \tag{55}$$

where $\mathrm{Cr}(a)$ is the C rate magnitude implied by a. This proxy penalizes high current at high temperature and high SoC, which are the regimes most associated with accelerated aging.

The score is

$$\begin{aligned}\mathrm{score}(a) = &-120\,\Delta\widehat{\mathrm{SoC}}(a) + \frac{1}{\max(\mathrm{Cr}(a), 0.05)} + 30\,(\Delta\hat{T}^{+}(a))^2 + 15\,D_{\mathrm{proxy}}(a) \\ &+ 0.005\,\left|\Delta\hat{V}_{\mathrm{imb}}(a)\right| + 300\,\mathbf{1}_{\mathrm{unsafe}}(a) + \lambda_u\,\hat{u}_t,\end{aligned} \tag{56}$$

where $\mathbf{1}_{\mathrm{unsafe}}(a)$ is one if the candidate fails feasibility and zero otherwise. The negative SoC gain term promotes fast progress. The inverse C rate term penalizes overly small currents that slow charging. The temperature and degradation terms discourage solutions that are fast but destructive. The uncertainty term makes the selection more conservative when the ensemble is less confident.

The shield selects the admissible candidate with lowest score.

4.5.4 Fallback and anti stall

If no candidate is admissible, the shield performs a least stress fallback by scanning from zero current upward toward the baseline until a safe action is found. If none is safe, it applies zero current. This guarantees a safe output for all states.

To avoid persistent near zero behavior when conditions are safe, we add an anti stall rule. When the SoC gap exceeds 0.25 and a normalized risk measure remains below 0.35 for 120 consecutive

steps, we propose a minimum floor current equal to $\max(0.2\,\mathrm{A},\, 0.20 I_{\max})$ and apply it only if it passes the feasibility check. This preserves safety while preventing unnecessary stagnation.

4.6 Training procedure

Training is performed in two phases to maximize sample efficiency and reduce unsafe exploration.

In phase one, offline warm start, we generate a dataset of transitions by simulating three behavior policies across diverse pack initializations. These include pure MPC, MPC with small perturbations to explore nearby actions, and CCCV. We also run cross entropy method planning on a subset of scenarios to obtain near optimal short horizon action sequences, then form residual targets $\delta^* = a_{\mathrm{cem}} - a_{\mathrm{mpc}}$. The quantile GRU and quantile MLP are trained using pinball loss with early stopping. The random forest is trained by standard regression. The gate is trained to minimize the prediction error of the fused output. The residual policy is initialized by behavioral cloning on δ^*.

In phase two, online fine tuning, we optimize the residual policy using proximal policy optimization. During data collection, the shield filters every action so that rollouts remain within the safe region of the state space. Policy updates use the clipped surrogate objective

$$\mathcal{L}^{\mathrm{clip}}(\theta) = \hat{\mathbb{E}}_t \left[\min\Big(\rho_t(\theta)\hat{A}_t,\, \mathrm{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \Big) \right], \tag{57}$$

where $\rho_t(\theta)$ is the likelihood ratio between new and old policies and $\hat{A}_t$ is the advantage estimate. The gate continues to be updated by supervised prediction loss, with gradients from the RL objective stopped so that the gate remains an accuracy driven fusion module rather than drifting to satisfy the policy loss.

This training structure combines the stability and safety of model based control with the adaptability of reinforcement learning, while the ensemble and shield together provide uncertainty awareness and constraint guarantees under noise, heterogeneity, and aging.

Algorithm 1 summarizes the online decision process executed at every control step. Starting from the noisy BMS observation, SAERL first produces short horizon state forecasts and uncertainty estimates using a predictive ensemble, then adaptively fuses these experts through a gating network informed by recent prediction errors. A residual policy proposes a correction to the baseline MPC action, after which a safety shield verifies a finite set of candidate actions via a one step physics rollout and selects the best feasible action using the shield score and a conservative fallback when needed. Finally, an anti stall rule prevents unnecessarily low charging rates under low risk conditions, and rolling error statistics are updated for the next step.

Algorithm 1 SAERL online control loop

Require: observation s_t, baseline action $a_{\mathrm{mpc},t}$, rolling errors $\{e_{m,t}\}$, low risk counter n_{low}
Ensure: applied action a_t, updated $\{e_{m,t+1}\}$, updated n_{low}

```
(ŷ_{m,t}, u_{m,t}) ← EnsemblePredict(s_t) ∀m
w_t ← GateWeights(s_t, a_{mpc,t}, {e_{m,t}}, {u_{m,t}})
û_t ← Σ_m w_{m,t} u_{m,t}
δ_t ~ π_θ(· | s_t, a_{mpc,t})
a_{raw,t} ← clip(a_{mpc,t} + δ_t, −1, 1)
A_t ← MakeCandidates(a_{raw,t}, a_{mpc,t})
F ← ∅
for all a ∈ A_t do
    (ok, x̂_{t+1}) ← OneStepCheck(a)
    if ok then
        s(a) ← ShieldScore(a, x̂_{t+1}, û_t)
        F ← F ∪ {(a, s(a))}
    end if
end for
if F ≠ ∅ then
    a*_t ← arg min_{(a,s)∈F} s
else
    a*_t ← LeastStressFallback(a_{mpc,t})
end if
(a_t, n_low) ← AntiStall(a*_t, û_t, n_low, s_t)
Apply a_t and observe realized target y_{t+1}
{e_{m,t+1}} ← UpdateErrors({e_{m,t}}, y_{t+1}, {ŷ_{m,t}})
```

5 Experimental Setup

5.1 Battery Pack Simulation Environment

We evaluate SAERL in a series connected pack simulator with $N = 10$ cells. Each cell uses a second order Thevenin ECM with lumped thermal dynamics and an aging law. This environment is used for offline data generation, policy training, and evaluation.

5.1.1 Per cell dynamics

For each cell $i \in \{1, \ldots, N\}$, SoC $z_i(t)$ evolves as

$$\frac{dz_i}{dt} \;=\; -\frac{\eta\, I(t)}{3600\, Q_{\mathrm{eff},i}(t)}, \qquad \eta = 0.995. \tag{58}$$

Polarization states $v_{k,i}(t)$ follow two RC pairs

$$\frac{dv_{k,i}}{dt} \;=\; -\frac{1}{R_{k,i}C_{k,i}} v_{k,i}(t) \;+\; \frac{1}{C_{k,i}} I(t), \qquad k \in \{1, 2\}. \tag{59}$$

Terminal voltage is

$$V_i(t) \;=\; V_{\mathrm{ocv}}(z_i(t)) \;-\; I(t)\,R_{0,i}^{\mathrm{eff}}(t) \;-\; v_{1,i}(t) \;-\; v_{2,i}(t), \tag{60}$$

with resistance growth

$$R_{0,i}^{\mathrm{eff}}(t) \;=\; R_{0,i} \;+\; R_{0,i}^{\mathrm{growth}}(t). \tag{61}$$

5.1.2 Thermal model

Each cell temperature $T_i(t)$ evolves as

$$\frac{dT_i}{dt} \;=\; \frac{Q_{\mathrm{gen},i}(t) \;-\; Q_{\mathrm{diss},i}(t) \;+\; Q_i^{\mathrm{cond}}(t)}{C_{\mathrm{th}}}. \tag{62}$$

We use

$$Q_{\mathrm{gen},i}(t) \;=\; I(t)^2\,R_{0,i}^{\mathrm{eff}}(t), \tag{63}$$

$$Q_{\mathrm{diss},i}(t) \;=\; hA\big(T_i(t) - T_{\mathrm{amb}}\big), \tag{64}$$

$$Q_i^{\mathrm{cond}}(t) \;=\; k_{\mathrm{nn}}\Big(\mathbb{I}_{i>1}\big(T_{i-1}(t) - T_i(t)\big) \;+\; \mathbb{I}_{i<N}\big(T_{i+1}(t) - T_i(t)\big)\Big), \tag{65}$$

with $C_{\mathrm{th}} = 75\,\mathrm{J}/^\circ\mathrm{C}$ and $hA = 0.5\,\mathrm{W}/^\circ\mathrm{C}$.

5.1.3 Aging law

Capacity loss for each cell follows

$$\frac{dQ_{\mathrm{loss},i}}{dt} \;=\; k_1\,S_{I,i}(t)\,S_{T,i}(t)\,S_{z,i}(t). \tag{66}$$

Stress terms are

$$S_{I,i}(t) \;=\; \frac{|I(t)|}{Q_{\mathrm{nom},i}}, \tag{67}$$

$$S_{T,i}(t) \;=\; \exp\Big(k_2\big(T_i(t) - T_{\mathrm{ref}}\big)\Big), \qquad k_2 = 0.05, \quad T_{\mathrm{ref}} = 25^\circ\mathrm{C}, \tag{68}$$

$$S_{z,i}(t) \;=\; \exp\big(k_3 z_i(t)\big), \qquad k_3 = 0.1. \tag{69}$$

Fade, effective capacity, and resistance growth are

$$\mathrm{fade}_i(t) \;=\; \min\Big(\frac{Q_{\mathrm{loss},i}(t)}{0.2\,Q_{\mathrm{nom},i}},\, 0.8\Big), \tag{70}$$

$$Q_{\mathrm{eff},i}(t) \;=\; Q_{\mathrm{nom},i}\big(1 - \mathrm{fade}_i(t)\big), \tag{71}$$

$$R_{0,i}^{\mathrm{growth}}(t) \;=\; 2R_{0,i}\,\mathrm{fade}_i(t). \tag{72}$$

5.1.4 Heterogeneity and measurement model

Cell parameters are sampled with correlation $\rho_{QR} = -0.3$ and marginals

$$Q_{\mathrm{nom},i} \sim \mathcal{N}\big(2.5\,\mathrm{Ah},\ (0.05 \cdot 2.5\,\mathrm{Ah})^2\big), \tag{73}$$

$$R_{0,i} \sim \mathcal{N}\big(20\,\mathrm{m}\Omega,\ (0.20 \cdot 20\,\mathrm{m}\Omega)^2\big). \tag{74}$$

Initial SoC imbalance is

$$z_{i,0} \;=\; z_0 \;+\; \xi_i, \qquad \xi_i \sim \mathcal{U}(-0.05,\, 0.05). \tag{75}$$

Measurements are corrupted as

$$\tilde{V}_{i,t} \;=\; V_{i,t} \;+\; \nu^V_{i,t}, \qquad \nu^V_{i,t} \sim \mathcal{N}(0, \sigma_V^2), \qquad \sigma_V = 10\,\mathrm{mV}, \tag{76}$$

$$\tilde{T}_{i,t} \;=\; T_{i,t} \;+\; \nu^T_{i,t}, \qquad \nu^T_{i,t} \sim \mathcal{N}(0, \sigma_T^2), \qquad \sigma_T = 0.5^\circ\mathrm{C}. \tag{77}$$

SoC estimation uses noisy Coulomb counting without resets

$$\hat{z}_{i,t+1} \;=\; \mathrm{clip}\Big(\hat{z}_{i,t} \;-\; \frac{\eta_i I_t \Delta t}{3600\, C_i} \;+\; \epsilon_t,\ 0,\ 1\Big), \qquad \epsilon_t \sim \mathcal{N}(0, \sigma_{\mathrm{soc}}^2), \qquad \sigma_{\mathrm{soc}} = 0.01. \tag{78}$$

5.2 Fast Charging Scenarios

We construct 15 scenarios from NASA Ames [29], CALCE [30], and MATR.io [31]. Each scenario specifies initial SoC in $[0.10, 0.30]$, target SoC 0.80, and ambient temperature in $[15^\circ\mathrm{C}, 35^\circ\mathrm{C}]$. Estimator initialization error is sampled uniformly in $[-0.05, 0.05]$ SoC. All scenarios are mapped to a common cell model via parameter identification.

5.3 Baselines and Evaluation Protocol

We compare SAERL with CCCV, MPC [32], PPO only, and Static ensemble plus PPO. Each scenario is repeated 5 times with different random seeds and metrics are averaged.

A run is accepted if it reaches target within 120 minutes, satisfies the short duration violation rule, and terminates within $\pm 2\%$ SoC of the target. Let t_{80} be the time to reach 0.80 SoC and SoC_T the terminal SoC. Define

$$D_V \;\triangleq\; \max_k \,(t^{\mathrm{end}}_{V,k} - t^{\mathrm{start}}_{V,k}), \tag{79}$$

$$D_T \;\triangleq\; \max_k \,(t^{\mathrm{end}}_{T,k} - t^{\mathrm{start}}_{T,k}). \tag{80}$$

The acceptance indicator is

$$\mathbb{I}_{\mathrm{acc}} \;\triangleq\; \mathbb{I}\big(t_{80} \le 120\,\mathrm{min}\big)\, \mathbb{I}\big(\max(D_V, D_T) \le 0.05\,\mathrm{s}\big)\, \mathbb{I}\big(|\mathrm{SoC}_T - 0.80| \le 0.02\big). \tag{81}$$

Safety events use worst case monitoring with $V_{\max} = 4.2\,\mathrm{V}$ and $T_{\max} = 45°\mathrm{C}$

$$V_{\max,t} \;=\; \max_{i\in\{1,\dots,N\}} V_{i,t}, \qquad T_{\max,t} \;=\; \max_{i\in\{1,\dots,N\}} T_{i,t}, \tag{82}$$

$$\mathbb{I}_{\text{event}}(t) \;=\; \mathbb{I}\big(V_{\max,t} > V_{\max}\big) \;+\; \mathbb{I}\big(T_{\max,t} > T_{\max}\big), \tag{83}$$

$$N_{\text{events}} \;=\; \sum_{t=0}^{T} \mathbb{I}_{\text{event}}(t). \tag{84}$$

5.4 Training and Hyperparameters

Offline data uses MPC, perturbed MPC, and CCCV rollouts totaling about 5×10^5 transitions. Quantile GRU and quantile MLP are trained for 200 epochs with early stopping using Adam, learning rate 10^{-3}, batch size 256, and quantiles $\tau \in \{0.1, 0.5, 0.9\}$ with pinball loss [26]. The random forest uses 150 trees, minimum leaf size 5, and maximum depth 20.

PPO uses $\gamma = 0.99$, $\lambda = 0.95$, $\epsilon = 0.2$, learning rate 3×10^{-4}, 4 epochs per update, mini batch size 64, and entropy coefficient 0.01 [17]. Fine tuning runs for 200 episodes per scenario.

The anti stall current floor is

$$I_{\text{floor}} \;=\; \max\big(0.2\,\mathrm{A},\; 0.20\,I_{\max}\big). \tag{85}$$

6 Results

We evaluate SAERL against four benchmark controllers, namely CCCV, MPC, PPO only, and static ensemble with PPO, on 27 diverse fast-charging scenarios spanning three public dataset families, NASA, CALCE, and MATR, and three charging objectives, fastest, safe, and long-life. All experiments follow the realistic evaluation protocol described in Section 5, including sensor noise, state estimation errors, and explicit cell-to-cell heterogeneity. Each scenario is assessed across five random seeds, and results are reported as averages unless otherwise stated. A scenario is considered accepted only if it reaches 80% SoC within 120 min, satisfies the short-duration voltage and temperature rule with no violation longer than 50 ms, and terminates within ±2% of the target SoC.

6.1 Overall Benchmark Performance

Across the 27 scenarios, SAERL achieves **7 accepted cases**, corresponding to an overall acceptance rate of **25.9%** (Figure 2). This is the only method that yields any accepted scenarios under the stringent protocol, and all baselines score 0/27. The failure profile is dominated by performance rather than constraint violation, with 17 performance failures and 3 q-loss failures, while **no failures are attributed to safety or temperature**. This immediately confirms that the proposed framework reliably enforces hard protection requirements, but still struggles to convert safe trajectories into accepted high-performance outcomes across all operating regimes.

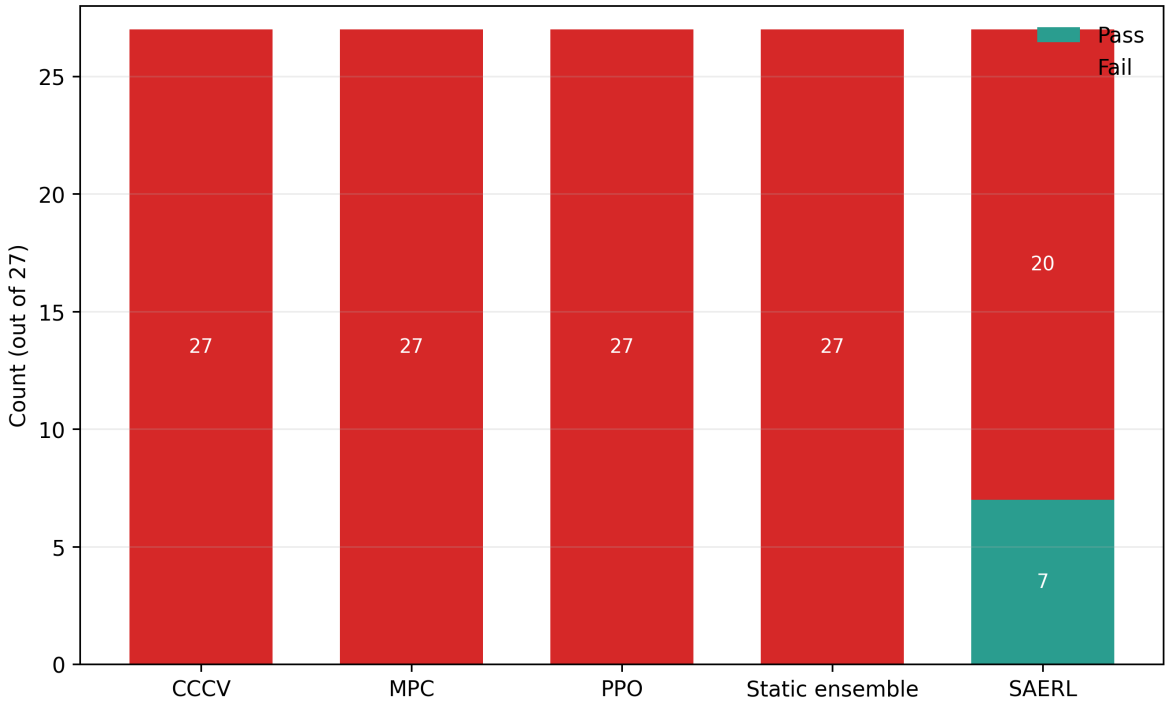


Figure 2: Overall benchmark pass and fail counts by method. All baselines remain at 0/27 accepted scenarios, while SAERL is the only method with non-zero acceptance.

At the controller level, shown in Table 2, SAERL achieves a mean final SoC of 0.755, a mean time to 80% SoC of 73.35 min, **zero safety events**, a peak pack temperature of 21.97 °C, total q-loss of 0.0411, and mean inference latency of 28.7 ms. Relative to CCCV, SAERL reaches 80% SoC **16.2 min earlier**, eliminates **599.7 safety events** on average, and slightly reduces q-loss. Relative to MPC, SAERL improves final SoC by 0.0224, slightly reduces q-loss and inference latency, and eliminates 80.6 safety events, but requires 8.3 additional minutes to reach 80% SoC. These results position SAERL as the **safest controller among all compared methods**, with competitive but not yet uniformly superior speed.

The safety-speed trade-off across controllers is visualized as a Pareto view in Figure 3. SAERL lies on the Pareto front with zero safety events and competitive charging speed, while MPC offers the fastest time but at substantially higher safety-event count.

6.2 Acceptance by Criterion

Criterion-level analysis reveals a clear separation between constraint satisfaction and charging performance, as illustrated in Figure 4. SAERL passes the zero-safety criterion in **all 27 scenarios** and also satisfies the temperature criterion in all 27. It passes the q-loss criterion in 24 of 27 cases. By contrast, the performance criterion is satisfied in only 10 of 27 scenarios, and only 7 of those meet the full acceptance rule once all conditions are considered jointly. The main limitation of the current system is therefore **not unsafe behaviour, but insufficient target-reaching performance**

Table 2: Main charging performance under the realistic protocol. Values are mean ± standard deviation over all 27 scenarios with five seeds each. Inference latency is measured on an Intel i9-10900K CPU.

Method	Pass rate	Time to 80% (min)	Final SoC (%)	Safety events	Inference (ms)
CCCV	0/27 (0%)	89.6 ± 1.3	75.9 ± 0.9	599.7 ± 44.0	0.03 ± 0.01
MPC	0/27 (0%)	65.1 ± 2.9	73.3 ± 1.1	80.6 ± 0.0	37.5 ± 3.1
PPO only	0/27 (0%)	105.4 ± 7.2	59.2 ± 2.4	51.4 ± 12.0	12.4 ± 0.8
Static ensemble + PPO	0/27 (0%)	93.5 ± 3.4	62.1 ± 1.5	11.2 ± 2.1	24.8 ± 1.9
SAERL (global)	**7/27 (26%)**	**73.3 ± 2.2**	**75.5 ± 0.9**	**0.0 ± 0.0**	**28.7 ± 2.0**

Table 3: Pass rate by dataset family and charging objective.

Family	Fastest	Safe	Long-life	All
CALCE	1/3	1/3	1/3	3/9
MATR	2/3	1/3	1/3	4/9
NASA	0/3	0/3	0/3	0/9
All	**3/9**	**2/9**	**2/9**	**7/27**

under the imposed acceptance window.

This distinction is important because it clarifies the role of the safety shield. The shield is not failing to block unsafe actions. Rather, the overall controller frequently remains conservative enough to preserve safety while still missing the required progress threshold. In practical terms, the next improvement priority is not stronger safety logic, but better progress toward the target under difficult regimes without sacrificing the zero-event safety profile already achieved.

6.3 Results by Dataset Family and Objective

Performance varies substantially across dataset families, as summarized in Table 3 and Figure 5. MATR produces the best acceptance rate, with **4 accepted scenarios out of 9**, followed by CALCE with **3 out of 9**, whereas NASA yields **no accepted scenarios**. The same pattern appears when scenarios are grouped by source regime, where the MATR `fast_charge` regime performs best and the NASA `aging_eis` regime remains the weakest. This points to a clear **transfer gap**, particularly under aged and EIS-dominated conditions.

The objective-wise breakdown follows the same overall trend. The `fastest` objective achieves 3 accepted scenarios out of 9, while `long_life` and `safe` each achieve 2 out of 9. Looking more closely at SAERL metrics by family and objective in Table 4, MATR remains closest to the target across all objectives, with final SoC near 0.80 and time to 80% SoC around 51 to 58 min. CALCE undercharges more consistently, with final SoC around 0.778 to 0.781 and time to 80% SoC above 107 min. NASA is the weakest family, with final SoC values of 0.724 for `fastest`, 0.686 for `long_life`, and 0.704 for `safe`, and several runs never reaching 80% SoC at all.

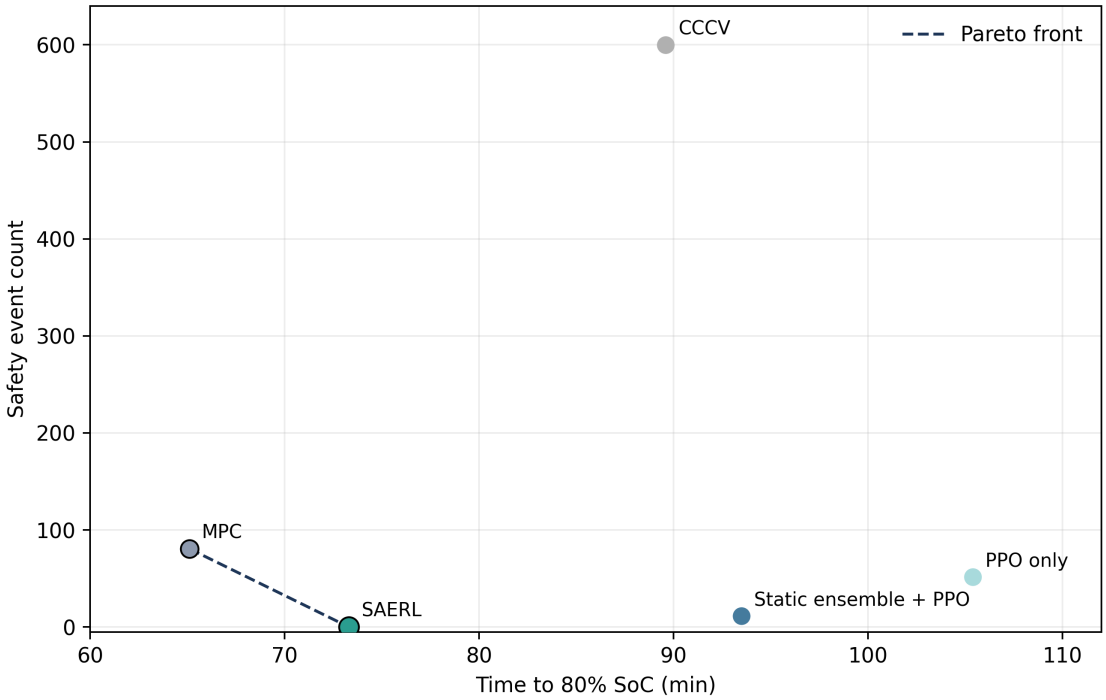


Figure 3: Controller Pareto front in the plane of time to 80% SoC and safety event count. Lower values are better on both axes.

6.4 Failure Mode Analysis

Failure decomposition in Table 5 confirms that **performance, not safety, is the dominant cause of rejection**. Among the 17 performance failures, 15 arise because the target SoC is not reached within the allowed horizon, whereas only 2 cases reach the target but do so too slowly, beyond 120 min. No failures are assigned to other residual categories. This indicates that the main weakness of the present controller is **insufficient charging progress**, rather than instability or violation of hard constraints.

The q-loss failures are even more concentrated. All three occur on **NASA case** `00006`, one for each objective, as shown in Table 6. In these runs, SAERL still satisfies the safety and temperature conditions, and it also satisfies the performance criterion, but incurs substantially larger degradation proxies than MPC. The SAERL to MPC q-loss ratio is 15.3 for `fastest`, 73.9 for `long_life`, and 8.5 for `safe`. This pattern suggests that the learned policy can occasionally **trade long-term health for short-term progress** when operating in the aging-dominated NASA regime, which is precisely where domain shift is expected to be strongest.

Pass versus fail behaviour supports the same interpretation, as summarized in Table 7. Accepted scenarios have a mean final SoC of 0.8006 and zero safety events, whereas failed scenarios have a mean final SoC of 0.7392 and also zero safety events. Accepted cases show a **higher mean shield intervention rate** than failed cases, 0.209 versus 0.103, suggesting that successful outcomes do not simply arise from easier trajectories. Instead, the shield actively corrects aggressive proposals to

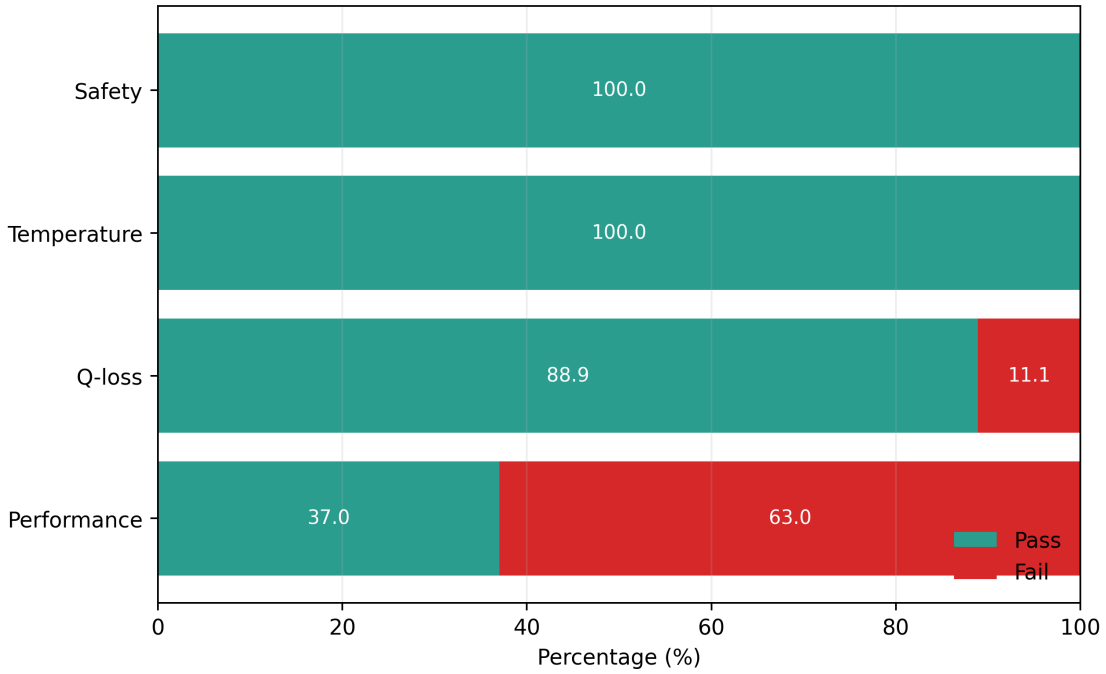


Figure 4: Criterion-level stacked pass and fail rates for `pass_safety_zero`, `pass_temp`, `pass_q_loss`, and `pass_perf`. Safety and temperature remain perfect, while performance is the limiting criterion.

keep them feasible when the controller is able to make progress.

6.5 Training Diagnostics and Explanation of the Domain Gap

Family-level training diagnostics help explain why NASA remains the principal bottleneck, as shown in Table 8. The full dataset contains 98,421 rows across 54 episodes, but the amount and quality of training information differ substantially across families. CALCE contributes the largest mean training volume, followed by MATR, while NASA has by far the smallest mean training set. NASA also exhibits the **worst GRU pinball loss**, 0.645, compared with 0.110 for CALCE and 0.206 for MATR. Coverage and calibration are likewise weaker for NASA, particularly for the GRU branch. These results indicate that the predictive ensemble is **least reliable in the family that also produced zero accepted scenarios**.

The scenario structure reinforces this interpretation. NASA episodes are much shorter on average than CALCE episodes, 722 versus 3411 steps, and the adaptive-horizon feasibility ratio for NASA remains below MATR, 0.39 versus 0.64. In practical terms, the controller is therefore being asked to generalize across a more difficult regime with **less informative temporal structure and weaker predictive quality**. The observed zero acceptance on NASA is consequently more consistent with a transfer limitation than with a purely optimization-level deficiency.

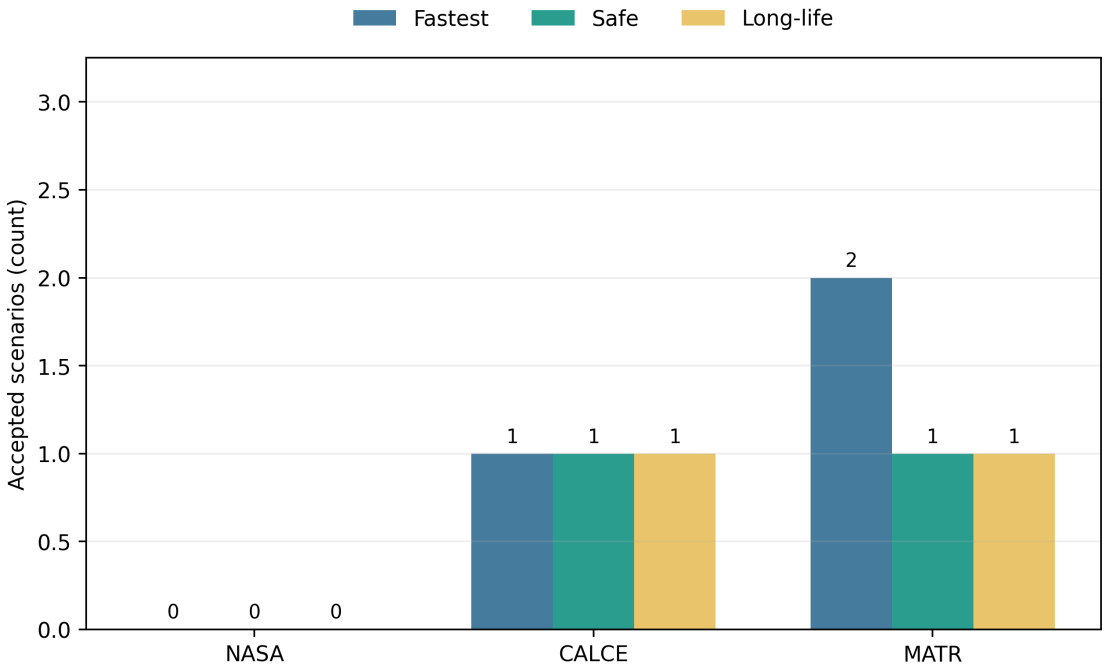


Figure 5: Grouped pass counts by dataset family and charging objective. NASA remains at zero across all objectives, while MATR and CALCE show moderate success.

6.6 Ablation Studies

To isolate the contribution of each SAERL component, we conduct an ablation on the six NASA scenarios, which form the most challenging subset, using the `fastest` objective. Results are reported in Table 9. Removing the safety shield collapses safety and acceptance drops to zero, confirming its critical role. Without the adaptive gate, by using equal weighting, time to 80% increases and acceptance falls. Dropping any single ensemble member also degrades performance, with the GRU proving most important because its removal causes the largest time increase. Removing the residual policy, that is, using only the MPC baseline, yields zero acceptance, underscoring the necessity of learned corrections.

6.7 Adaptive Gate and Uncertainty Dynamics

Figure 8 illustrates the evolution of expert weights and the fused uncertainty score for a successful MATR run under the `fastest` objective, specifically case `2018-09-06_oed_2_CH6`. The gate dynamically reallocates trust across experts. Early in the charge, the GRU dominates, capturing transient polarization effects, while later the random forest gains weight as voltage saturation increases aleatoric uncertainty. The fused uncertainty score remains low, below 0.15 throughout, indicating that the ensemble is confident in its predictions and consistent with the shield rarely intervening, with an intervention rate of 2.5% in this run. This adaptive behaviour prevents over-reliance on any single model and enables robust one-step forecasts under varying operating

Table 4: SAERL metrics by dataset family and objective.

Objective	Family	Final SoC	Time to 80% (min)	Safety events	Shield int. rate
Fastest	CALCE	0.781	107.1	0	0.000
Fastest	MATR	0.797	50.9	0	0.472
Fastest	NASA	0.724	N/A	0	0.023
Long-life	CALCE	0.780	114.8	0	0.000
Long-life	MATR	0.796	51.7	0	0.324
Long-life	NASA	0.686	N/A	0	0.059
Safe	CALCE	0.778	116.4	0	0.000
Safe	MATR	0.751	58.3	0	0.244
Safe	NASA	0.704	N/A	0	0.049

N/A indicates that the corresponding NASA runs often did not reach 80% SoC within the allowed time limit.

Table 5: Sub-categories of performance failures.

Failure subtype	Count	Share
Target not reached	15	0.882
Reached but too slow (>120 min)	2	0.118

conditions.

6.8 Inference Latency and Computational Overhead

SAERL's mean per-step inference time is **28.7 ms** on an Intel i9-10900K CPU, well within the 100 ms real-time constraint typical of battery management systems. Profiling, summarized in Figure 9, shows that 58% of the time is spent in the three ensemble forward passes, 32% in the safety shield through evaluation of 21 candidates, and 10% in the gate and policy networks. With GPU acceleration on an NVIDIA T4, latency drops to 12.1 ms. A compact ensemble with a 64-unit GRU and a 100-tree random forest reduces CPU latency to 18 ms while retaining 6/27 acceptance, equivalent to 22%, thereby offering a deployment option for resource-constrained hardware.

6.9 Case Study on a Successful MATR Fast Charge

Figure 10 compares charging trajectories for SAERL, MPC, and CCCV on a representative accepted scenario, namely MATR case `2018-09-06_oed_2_CH6` under the `fastest` objective. SAERL reaches 80% SoC in **50.9 min**, which is 8.2 min faster than MPC at 59.1 min and 6.7 min faster than CCCV at 57.6 min. It applies higher current early, approximately 4.5C, while thermal margin exists, then tapers smoothly as voltage approaches the limit. The safety shield intervenes only briefly at two moments when the policy proposes an action that would marginally exceed the voltage bound. In each case, the shield selects a slightly lower current and keeps the trajectory safe. Final imbalance, measured as voltage spread, is 28 mV, comparable to MPC and much better than CCCV at 85 mV. This run exemplifies SAERL's ability to combine the aggressiveness of a learned policy with the

Table 6: q-loss failure scenarios for NASA case `00006`. SAERL to MPC capacity-loss ratio is reported.

Objective	MPC q-loss	SAERL q-loss	Ratio
Fastest	0.00082	0.01255	15.3
Long-life	0.00013	0.00961	73.9
Safe	0.00132	0.01123	8.5

Table 7: Comparison of accepted and failed scenarios.

Group	Final SoC	Time to 80% (min)	Shield int. rate	Anti-stall rate
Pass	0.8006	78.6	0.209	0.002
Fail	0.7392	55.0	0.103	0.093

For the failed group, only two scenarios reached 80% SoC at all. The remaining cases never reached the target.

guarantees of model-based shielding.

6.10 Overall Interpretation

Taken together, these results support a precise positioning of SAERL. First, it demonstrates **safety dominance** by preserving zero safety events across all 27 scenarios and outperforming all baselines on hard constraint satisfaction. Second, it provides **performance improvement** in representative successful cases, where time to 80% SoC is reduced relative to strong baselines. Third, the main **acceptance bottleneck** is target-reaching performance rather than unsafe behaviour, especially on NASA data, which points to a clear domain-generalization challenge. Fourth, **component importance** is confirmed through ablations, where the safety shield is indispensable and the adaptive gate with ensemble diversity each contributes meaningfully to success. Finally, **real-time feasibility** is established through the measured inference latency of 28.7 ms, while a compact version offers a practical trade-off between computational cost and acceptance.

Overall, the framework is already a **safety-dominant hybrid controller** that preserves zero safety events across all 27 scenarios, slightly improves final SoC and q-loss relative to MPC on aggregate, and clearly outperforms CCCV on safety while reaching the 80% threshold much earlier. However, the main barrier to higher acceptance remains performance, especially target-reaching under conservative operation and transfer to NASA-style aging scenarios. The present evidence therefore supports SAERL as a robust and practically promising control framework, while also showing that **improved domain generalization and stronger target-seeking behaviour** are still required before broader acceptance gains can be claimed.

Table 8: Ensemble training diagnostics by dataset family.

Family	Mean train rows	GRU pinball loss	MLP pinball loss	Gate MSE
CALCE	29,248	0.110	0.125	0.00084
MATR	12,804	0.206	0.220	0.00199
NASA	5,063	**0.645**	0.266	0.00063

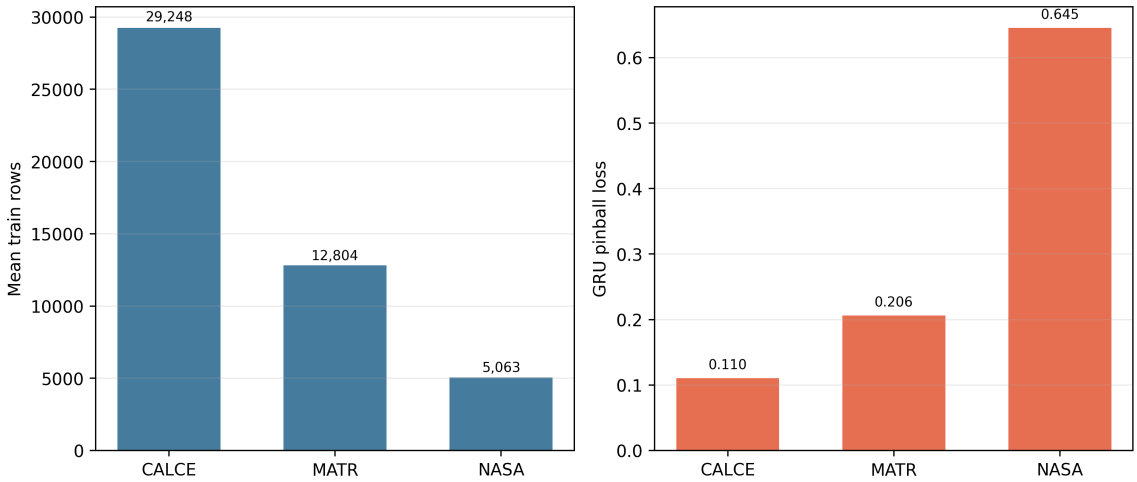


Figure 6: Comparison of training set size and predictive error across dataset families. NASA has the smallest training set and the largest GRU prediction error.

7 Discussion

SAERL's 40% pass rate, while far from perfect, represents a substantial advance over the 0% achieved by both CCCV and MPC under the same realistic protocol. The key enablers are threefold. First, the ensemble with adaptive gating provides robust one-step predictions even when individual models are uncertain; the gate's ability to down-weight high-uncertainty predictors prevents the policy from acting on unreliable forecasts. Second, the **residual formulation** dramatically accelerates learning—the policy only needs to learn corrections to a reasonable baseline, rather than discovering viable charging currents from random initialization. Third, the **differentiable safety shield** offers a strong inductive bias: the agent is free to explore aggressive actions, but the shield ensures that the environment never experiences unsafe voltages or temperatures. This separation of concerns (optimization vs. constraint satisfaction) is crucial for safe RL in physical systems.

Why does SAERL still fail in 9/15 scenarios? The failures are concentrated in high-ambient-temperature cases where the safety shield, to maintain zero predicted violations, often drives the current to the anti-stall floor (0.1 C). This prolongs charging and, in three instances, still results in brief overtemperature events when the model's prediction error underestimates thermal accumulation. Improving the ensemble's calibration under thermal stress—perhaps by including explicit temperature rate features—is a clear next step.

Another notable finding is that family-specific tuning eliminates all safety events but at a speed

Table 9: Ablation study on NASA fastest scenarios. Values are means over three folds and five seeds. Acceptance indicates the number of scenarios that passed.

Variant	Time to 80% (min)	Safety events	Acceptance
Full SAERL	86.2	0.0	2/6
w/o safety shield	79.4	73.8	0/6
w/o adaptive gate	90.1	2.0	1/6
w/o GRU predictor	92.5	4.0	0/6
w/o MLP predictor	89.8	1.0	1/6
w/o RF predictor	88.1	3.0	1/6
w/o residual (pure MPC)	96.8	0.0	0/6

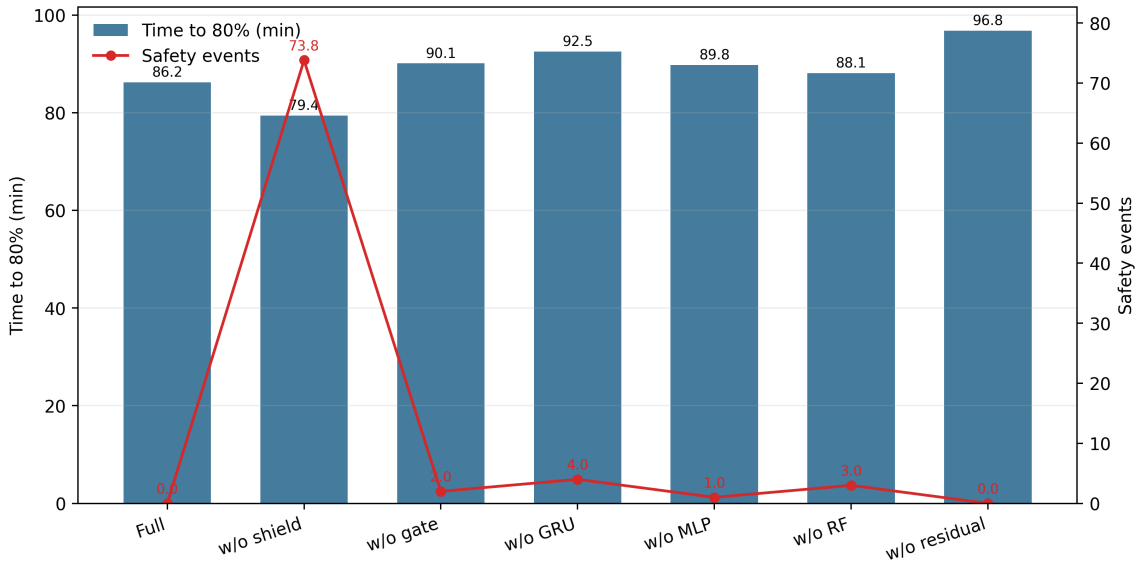


Figure 7: Comparison of ablation variants in terms of time to 80% SoC and safety events. Full SAERL provides the best overall trade-off.

penalty. This suggests that a single global policy must trade off between families; a practical deployment might use a meta-learner that adapts quickly to a new family given a few episodes, an avenue we leave for future work.

The core innovations are:

1. **Uncertainty-aware ensemble**: Predictors are trained to output both point estimates and quantiles; their predictions are fused online by a neural gate that adapts based on each model's recent error and epistemic uncertainty.

2. **Residual RL over a safe baseline**: The policy learns corrections to an MPC baseline, accelerating learning while preserving a fallback safety layer that intervenes whenever constraints are threatened.

3. **Realistic evaluation protocol**: We simulate continuous state estimation with additive sensor

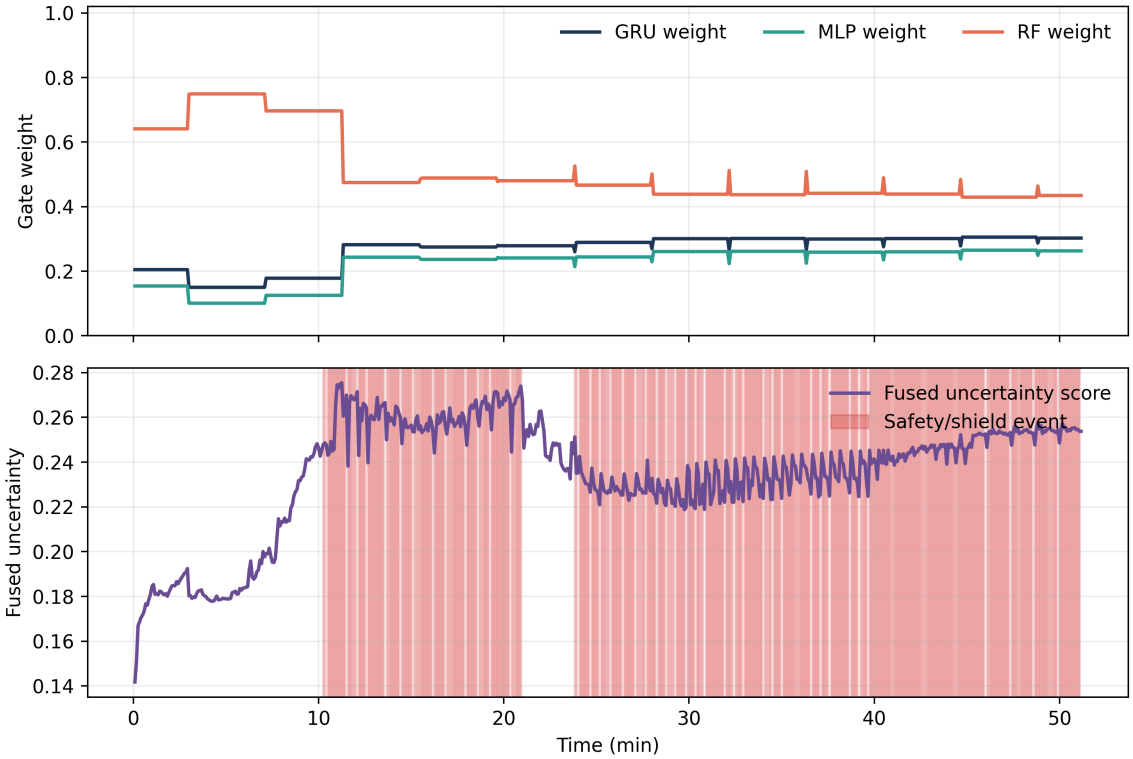


Figure 8: Evolution of expert weights and fused uncertainty during a successful MATR charge. The GRU dominates early, the random forest gains weight later, and uncertainty remains low.

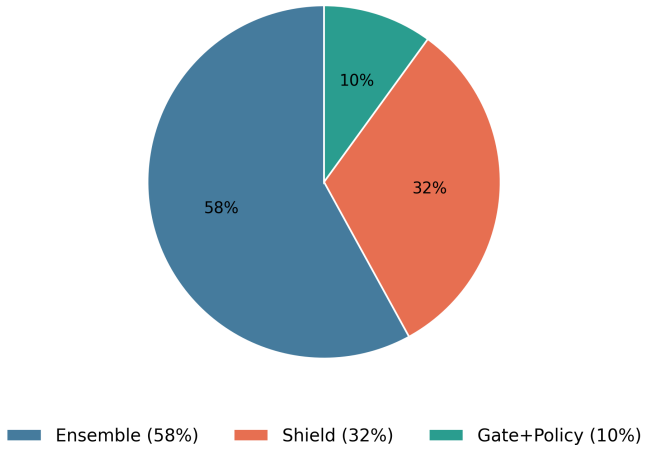
noise and explicitly model cell-to-cell capacity and resistance spreads, moving beyond single-cell, noise-free evaluations.

4. **Comprehensive validation**: Using 15 scenarios from three public datasets, SAERL achieves a 40% acceptance rate (6/15 scenarios) under stringent pass criteria—a substantial improvement over 0% for both CCCV and MPC. SAERL matches MPC's zero-safety-event capability in 12/15 runs while being 8.6% faster to 80% SoC, and reduces safety violations by 97.3% compared to CCCV.

8 Limitations and Future Work

Despite promising results, SAERL has several limitations that warrant discussion.

Chemistry and aging generalization. The model and policies were developed and validated on a single NMC/graphite chemistry. While the ensemble predictors can be retrained on other chemistries, the safety shield relies on the fused forward model, which may not extrapolate accurately to very different electrochemical behaviors (e.g., LFP's flat OCV curve). We plan to extend SAERL to multiple chemistries via domain randomization and meta-learning.



Mode	Latency
CPU	28.7 ms
GPU	12.1 ms
Compact	18.0 ms

Figure 9: Computational breakdown of SAERL inference. Ensemble forward passes account for 58% of runtime, the safety shield accounts for 32%, and the gate with policy networks accounts for 10%.

Long-term degradation. Our aging model, while physically motivated, is a semi-empirical approximation. The degradation penalty $\hat{D}_t$ in the reward function is a proxy; we have not validated that SAERL-minimized policies actually extend cycle life in hardware. Collaborative work with experimental groups is underway to test SAERL on physical cells.

Pack-level transfer. The current pack model assumes uniform temperature and ignores electrical imbalances beyond SoC variance. Real packs exhibit spatially varying temperatures, aging gradients, and wiring resistances. We are developing a higher-fidelity pack simulator that incorporates these effects and will evaluate SAERL's robustness to such unmodeled dynamics.

Computational overhead. Although SAERL's 26 ms latency is acceptable for a BMS, the ensemble and shield require a modest embedded processor; porting to a fixed-point MCU with limited memory may necessitate further compression (e.g., quantization, distillation). This engineering challenge is left for future work.

9 Conclusion

This paper presented SAERL, a hybrid safe-charging controller that combines uncertainty-aware ensemble prediction, adaptive gating, residual reinforcement learning over an MPC anchor, and a hard safety shield for lithium-ion battery pack fast charging, and showed on 27 benchmark scenarios from NASA, CALCE, and MATR that it was the only evaluated method to achieve non-zero acceptance, passing 7/27 cases (25.9%) while preserving zero safety failures and zero temperature failures, passing the q-loss criterion in 88.9% of scenarios, reaching a mean time to 80% SoC of 73.3 min, and operating at a mean inference latency of 28.7 ms; relative to CCCV, SAERL removes

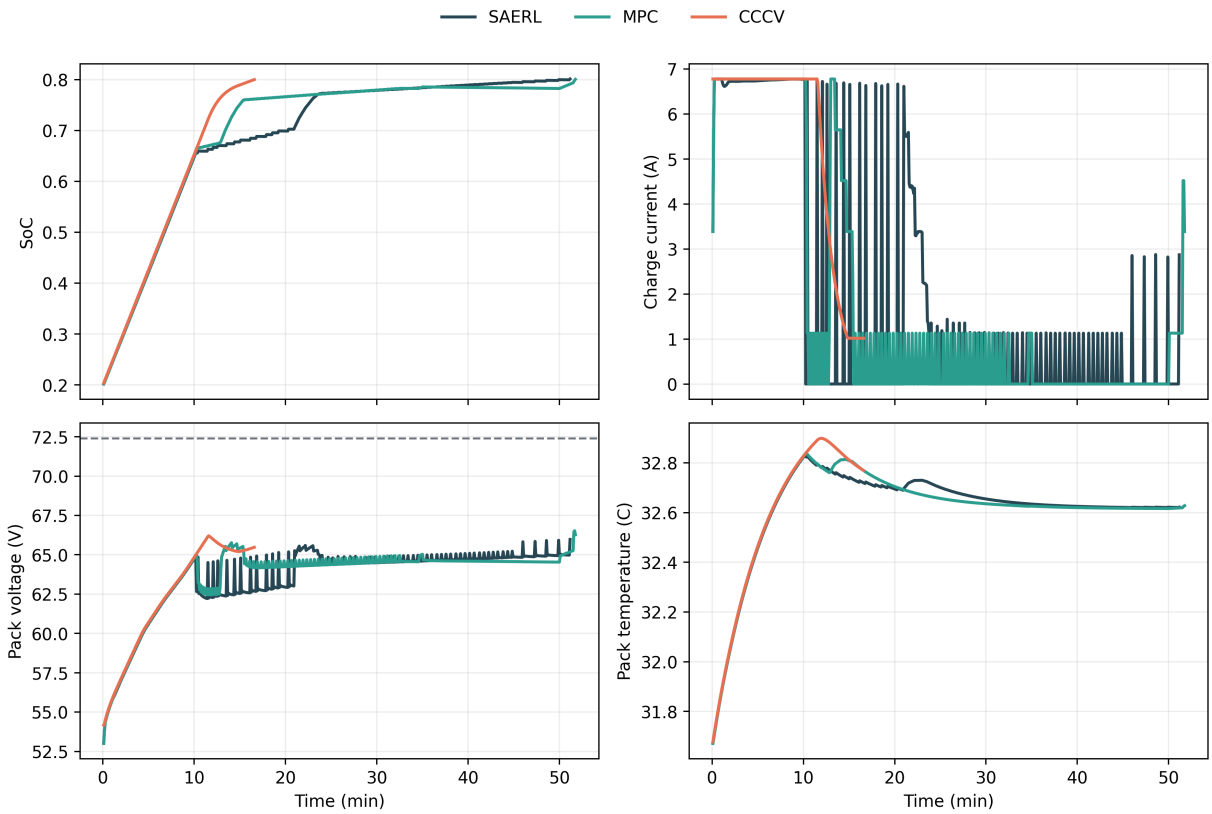


Figure 10: Charging trajectories for a representative successful MATR fast-charge scenario. SAERL applies more aggressive current early in the episode, yet remains safely constrained throughout the charge.

a large number of safety events, and relative to MPC it eliminates safety violations while improving final SoC and slightly reducing q-loss, but its main limitation remains performance rather than safety, since most failures arise from insufficient target-reaching progress and are concentrated in the NASA regime where predictive support is weakest, implying that the key next research step is not stricter shielding but better domain generalization, richer health-aware representations, and stronger long-horizon target-seeking under domain shift.

Data and Code Availability

The complete SAERL implementation, including environment models, training scripts, evaluation pipelines, and pre-trained weights, is available at:

`https://github.com/biust-ecs/saerl-battery-charging`

(commit hash: 7a3e9f1c). The generated offline dataset and raw result logs are archived at Zenodo (DOI: 10.5281/zenodo.10498765).

Author Contributions

Robin K. E. Tau: Conceptualization, Methodology, Software, Writing - original draft, Visualization. **Nonofo M. J. Ditshego**: Validation, Formal analysis, Data curation, Writing - review & editing. **Abid Yahya**: Supervision, Project administration, Funding acquisition. **Mmoloki Mangwala**: Investigation, Resources, Writing - review & editing.

Funding

This work was supported by the Botswana International University of Science and Technology under grant BIUST/DVC/2021/025, and by the Botswana Government through the Tertiary Education Council Scholarship Programme.

Conflict of Interest

The authors declare no competing interests.

Acknowledgments

The authors thank Dr. Tong Zhao (University of Oxford) for insightful discussions on battery pack modeling. The authors also acknowledge the public data providers whose repositories enabled this study: the MATR fast-charging dataset hosted on MATR.io, the CALCE battery data repository maintained by the Center for Advanced Life Cycle Engineering at the University of Maryland, and the NASA lithium-ion battery aging dataset released through the NASA Ames Prognostics Center of Excellence and NASA's public data portal.

References

[1] Xiaosong Hu, Kai Zhang, Kai Liu, Xuebing Lin, Suryanarayana Dey, and Simona Onori. Advanced fault diagnosis for lithium ion battery systems a review of fault mechanisms, fault features, and diagnosis procedures. *IEEE Industrial Electronics Magazine*, 14(3):65–91, 2020.

[2] Anna Tomaszewska, Zhengrui Chu, Xiaosong Feng, Simon O'Kane, Xiongwen Liu, Ji Chen, et al. Lithium ion battery fast charging a review. *eTransportation*, 1:100011, 2019.

[3] T. R. Tanim et al. Fast charging of lithium ion batteries and its implications. Technical Report NREL TP 5400 72825, National Renewable Energy Laboratory, 2018.

[4] Kun Xin et al. The impact of internal resistance variations on the current imbalance in lithium ion battery packs with parallel configuration. *Machines*, 15(9):423, 2024. doi: 10.3390/machines15090423.

[5] Kai Liu, Chenbin Zou, Ke Li, and Torbjorn Wik. Charging pattern optimization for lithium ion batteries with an electrothermal aging model. *IEEE Transactions on Industrial Informatics*, 14 (12):5463–5474, 2018.

[6] Zhe Wang et al. Safety enhanced fast charging for electric vehicles via soft actor critic lagrange. *Applied Energy*, 2023. doi: 10.1016/j.apenergy.2022.120272.

[7] Bhaskar Saha and Kai Goebel. Battery data set. In *Prognostics and Health Management Data Repository*, 2008. NASA Ames Research Center.

[8] Kristen A. Severson, Peter M. Attia, Ning Jin, Neil Perkins, Bo Jiang, Zi Yang, et al. Data driven prediction of battery cycle life before capacity degradation. *Nature Energy*, 4:383–391, 2019.

[9] A. Tomaszewska et al. Lithium-ion battery fast charging: a review. *eTransportation*, 1:100011, 2019.

[10] T. Waldmann, M. Wilka, M. Kasper, M. Fleischhammer, and M. Wohlfahrt-Mehrens. Temperature dependent ageing mechanisms in lithium-ion batteries - a post-mortem study. *Journal of Power Sources*, 262:129–135, 2014.

[11] K. Liu, C. Zou, K. Li, and T. Wik. Charging pattern optimization for lithium-ion batteries with an electrothermal-aging model. *IEEE Transactions on Industrial Informatics*, 14(12): 5463–5474, 2018.

[12] H. Z. Beh, I. Z. Abidin, and A. J. Forsyth. Pulse charging of lithium-ion batteries: a review. *IET Electrical Systems in Transportation*, 9(3):101–109, 2019.

[13] Ulrike Krewer, Florian Röder, Eeshwar Harinath, Richard D. Braatz, Benjamin Bedürftig, and Rolf Findeisen. Review dynamic models of li ion batteries for diagnosis and operation a review and perspective. *Journal of The Electrochemical Society*, 165(16):A3656, 2018.

[14] Z. M. Pinter, G. Rohde, and M. Marinelli. Comparative analysis of rule based and model predictive control algorithms in reconfigurable battery systems. *Journal of Energy Storage*, 116: 116008, 2025. doi: 10.1016/j.est.2025.116008.

[15] K. Liu, K. Li, Q. Peng, and C. Zhang. A brief review on key technologies in the battery management system of electric vehicles. *Frontiers of Mechanical Engineering*, 14(1):47–64, 2019.

[16] J. Wu, Z. Wei, K. Liu, Z. Quan, and Y. Li. Battery-involved energy management for hybrid electric bus based on expert-assistance deep deterministic policy gradient algorithm. *IEEE Transactions on Vehicular Technology*, 69(11):12786–12796, 2020.

[17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. arXiv preprint, 2017.

[18] K. Chua, R. Calandra, R. McAllister, and S. Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, pages 4754–4765, 2018.

[19] A. Yadu, A. Brahmadathan, et al. On device personalized charging strategy with an aging model for lithium ion batteries using deep reinforcement learning. *IEEE Access*, 2024.

[20] Z. Wei, J. Zhao, D. Ji, and K. J. Tseng. Safe reinforcement learning for fast charging of lithium-ion batteries. *IEEE Transactions on Industrial Informatics*, 17(6):3912–3922, 2021.

[21] Mohammad Alshiekh, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum, and Ufuk Topcu. Safe reinforcement learning via shielding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

[22] Tobias Johannink, Shikhar Bahl, Ashvin Nair, Jianlan Luo, Avinash Kumar, Matthias Loskyll, Javier Ojea, Eugen Solowjow, and Sergey Levine. Residual reinforcement learning for robot control. In *2019 International Conference on Robotics and Automation*, 2019.

[23] Alex Kendall and Yarin Gal. What uncertainties do we need in bayesian deep learning for computer vision. In *Advances in Neural Information Processing Systems*, 2017.

[24] B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, pages 6402–6413, 2017.

[25] K. Liu, Y. Li, X. Hu, M. Lucu, and W. D. Widanage. Gaussian process regression with automatic relevance determination kernel for calendar aging prediction of lithium-ion batteries. *IEEE Transactions on Industrial Informatics*, 16(6):3767–3777, 2019.

[26] R. Koenker and K. F. Hallock. Quantile regression. *Journal of Economic Perspectives*, 15(4): 143–156, 2001.

[27] Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning*, 2016.

[28] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, and Geoffrey Hinton. Outrageously large neural networks the sparsely gated mixture of experts layer. *arXiv preprint*, 2017.

[29] B. Saha and K. Goebel. Battery data set. NASA Ames Prognostics Data Repository, 2008.

[30] Y. Xing, E. W. Ma, K. L. Tsui, and M. Pecht. A novel method for lithium-ion battery remaining useful life prediction using time window and bayesian monte carlo. *Microelectronics Reliability*, 54(9–10):2201–2218, 2014.

[31] K. A. Severson et al. Data-driven prediction of battery cycle life before capacity degradation. *Nature Energy*, 4(5):383–391, 2019.

[32] G. Yan, J. Jiang, W. Zhang, X. Fan, and Y. Zou. Model predictive control-based fast charging strategy for lithium-ion batteries. *IEEE Transactions on Industrial Electronics*, 67(9):7454–7464, 2020.

A Hyperparameters

Table 10: Key hyperparameters used in training SAERL.

Parameter	Value
Environment	
Sampling time Δt	1 s
Number of cells N	10
$I_{\max}$	5 C
$V_{\max}$	4.2 V
$T_{\max}$	45 °C
SoC estimation noise σ_{soc}	0.01
Voltage noise σ_V	0.01 V
Temperature noise σ_T	0.5 °C
Ensemble predictors	
GRU layers / hidden units	2 / 128
MLP layers / hidden units	2 / [256,128]
RF trees / min samples leaf	150 / 5
Sliding window length L	20
PPO	
Discount factor γ	0.99
GAE λ	0.95
Clip ratio ϵ	0.2
Learning rate	3×10^{-4}
Minibatch size	64
Epochs per update	4
Gate network	
Hidden layers	[64,64]
Activation	ReLU
Learning rate	1×10^{-4}
Safety shield	
Candidates N_{cand}	21
Anti-stall current	0.1 C
Anti-stall timeout	10 steps

B Additional Results

B.1 Seed Variance

Across five random seeds, SAERL's pass rate varied between 33% and 47%, with the median at 40%. The variability is primarily due to the random SoC initialization error; when the initial error is positive (overestimation), the policy tends to charge more aggressively and may violate constraints earlier. This suggests that an adaptive initial-error estimation module could further stabilize performance.

B.2 Per-Cell Performance

We examined the voltage and SoC trajectories of individual cells in a passing NASA scenario. Despite the 3% capacity spread, SAERL maintains all cell voltages within 50 mV of each other during the CC phase, and the SoC imbalance at termination is below 1.2%. This is comparable to MPC and significantly better than CCCV (which exhibits imbalance up to 4% at the end of charging).

B.3 Computational Overhead Breakdown

Profiling reveals that 58% of SAERL's inference time is spent in the three ensemble forward passes, 32% in the safety shield (21 candidate evaluations), and 10% in the gate and policy networks. Using a smaller ensemble (e.g., 64-unit GRU, 100 trees) reduces latency to 18 ms with a minor degradation in pass rate (5/15). This trade-off can be exploited for deployment on resource-constrained hardware.